// ===== src/cli/index.ts =====

```
#!/usr/bin/env node
/**
 * 飞虹 Code (Muse Code 参照复刻)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * CLI 入口：参数解析 → 版本/帮助/单命令/REPL/管理命令 分发
 */
import { randomUUID } from 'crypto';
import { readFileSync, existsSync } from 'fs';
import { setRunId } from '../shared/logger';
import { AppError } from '../shared/errors';
import { loadDotEnv } from '../shared/config';
import { parseArgs, type SkillCommand, type ManagementCommand } from './commands';
import { startRepl } from './repl';
import {
  runGoal,
  isOfflineByDefault,
  runPlanSkill,
  runGrillSkill,
```

```
20  runGoalSkill,
21  runSelfHealSkill,
22  runParallelGoal,
23  runSessions,
24  runResume,
25  runDiff,
26  runRollback,
27  runWhoami,
28  runPolicyCmd,
29  runAudit,
30  runAuditVerify,
31  runTenants,
32  runDoctor,
33  runPluginCmd,
34  runSkillMarketCmd,
35  runReviewCmd,
36  runTeamCmd,
37  runServe,
38  runCodeWrite,
39  runQualityGate,
40  runSelfImprove,
```

```
41   runSwe,
42   runModelStats,
43   runExperiences,
44   runHarness,
45 } from './run';
46 import { VERSION } from './version';
47 import { setLang, t } from '../shared/i18n';
48
```

```
function printVersion(): void {
  console.log(`fhcode v${VERSION}`);
  console.log(`${t('app.product')} — ${t('app.tagline')}`);
  console.log(t('app.signature'));
}

function printHelp(): void {
  console.log(t('cli.help', { version: VERSION, signature: t('app.signature') }));
}

/** M1.3: 读取上下文文件内容（限 32KB；不存在/不可读返回 null） */
const MAX_CONTEXT_BYTES = 32 * 1024;
function readContextFile(path: string): string | null {
  try {
    if (!existsSync(path)) return null;
    const buf = readFileSync(path);
    return buf.subarray(0, MAX_CONTEXT_BYTES).toString('utf8');
  } catch {
    return null;
  }
}
```

```
70
71 async function main(): Promise<void> {
72   setRunId(randomUUID());
73   loadDotEnv(); // 尽早加载 .env（须在 isOfflineByDefault / loadConfig 之前）
74
75   const args = parseArgs(process.argv.slice(2));
76   setLang(args.flags.lang);
77
78   if (args.flags.version) {
79     printVersion();
80     return;
81   }
82   if (args.flags.help) {
83     printHelp();
84     return;
85   }
86   if (args.skill) {
87     await dispatchSkill(args.skill);
88     return;
89   }
90   if (args.manage) {
```

```
    await dispatchManage(args.manage);
    return;
  }
  if (args.flags.parallel && args.command) {
    await runParallelGoal(args.command);
    return;
  }
  if (args.command) {
```

```
99      const offline = isOfflineByDefault();
100     let goal = args.command;
101     // M1.3: --context-file 附加活动文件/选区文件内容作为结构化上下文（限 32KB 防爆上下文）
102     if (args.flags.contextFile) {
103       const content = readContextFile(args.flags.contextFile);
104       if (content !== null) {
105         goal += `\n\n<context-file: ${args.flags.contextFile}>\n${content}\n</context-file>`;
106       }
107     }
108     await runGoal(goal, { offline, stream: args.flags.stream });
109     return;
110   }
111
112   await startRepl();
113 }
114
115 async function dispatchSkill(skill: { kind: SkillCommand; arg: string }): Promise<void> {
116   if (skill.kind === 'plan') console.log(runPlanSkill(skill.arg || ''));
117   else if (skill.kind === 'grill') console.log(runGrillSkill(skill.arg || '.'));
118   else if (skill.kind === 'self-heal') console.log(runSelfHealSkill(skill.arg || ''));
119   else console.log(runGoalSkill(skill.arg || ''));
```

```
}

async function dispatchManage(m: ManagementCommand): Promise<void> {
  switch (m.kind) {
    case 'sessions': await runSessions(); break;
    case 'resume': await runResume(m.id); break;
    case 'diff': await runDiff(m.id); break;
    case 'rollback': await runRollback(m.id, m.yes); break;
    case 'whoami': runWhoami(); break;
    case 'policy': runPolicyCmd(); break;
    case 'audit': if (m.verify) runAuditVerify(); else runAudit(m.limit); break;
    case 'tenants': runTenants(); break;
    case 'doctor': await runDoctor(); break;
    case 'plugin': await runPluginCmd(m.action, m.source); break;
    case 'skill-market': await runSkillMarketCmd(m.action, m.query, m.market); break;
    case 'review': runReviewCmd(m.path, m.json); break;
    case 'team': await runTeamCmd(m.goal); break;
    case 'serve': runServe(m.port); break;
    case 'code-write': runCodeWrite(m.goal); break;
    case 'quality-gate': runQualityGate(m.path); break;
    case 'self-improve': await runSelfImprove(); break;
```

```
141     case 'model-stats': runModelStats(); break;
142     case 'experiences': runExperiences(m.path); break;
143     case 'harness': await runHarness({
144       split: m.split,
145       limit: m.limit,
146       offset: m.offset,
147       mode: m.mode,
148       report: m.report,
```

```
149       json: m.json,
150     }); break;
151     case 'swe': await runSwe(m.goal, {
152       repo: m.repo,
153       maxTasks: m.maxTasks,
154       maxRetries: m.maxRetries,
155       maxIterations: m.maxIterations,
156       verifyOnly: m.verifyOnly,
157       planOnly: m.planOnly,
158     }); break;
159   }
160 }
161
162 main().catch((err: unknown) => {
163   setRunId(randomUUID());
164   if (err instanceof AppError) {
165     console.error(t('err.runFailed', { code: err.code, message: err.message }));
166   } else {
167     console.error(t('err.unexpected') + (err instanceof Error ? err.message : String(err)));
168   }
169   console.error(t('err.hint'));
```

```
  process.exit(1);
});
```

```
// ===== src/cli/version.ts =====
```

```
/**
 * 飞虹 Code (Muse Code 参照复刻)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 版本号集中定义
 */
export const VERSION = '0.6.0';
export const PRODUCT = '飞虹 Code';
export const TAGLINE = '终端 AI 编程智能体（Muse Code 参照复刻）';
export const SIGNATURE =
  '晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹';

```

```
// ===== src/agent/orchestrator.ts =====
```

```
/**
 * 飞虹 Code (Muse Code 参照复刻)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 编排器：ReAct 循环（模型 → 工具调用 → 结果回填 → 完成）。
 * 依赖注入 ModelRouter / ToolRegistry / EventLog / SessionStore，便于测试与替换。
 *
 * M3 增强：
```

```
 *  - run(goal, resume?) 支持从检查点续跑（中断任务恢复）
 *  - 每轮迭代后通过 persist 回调落盘检查点（含完整对话、迭代数、成本、被改文件）
 *
 * M6 增强：
 *  - 自我修复循环：错误自动识别 + 反思重试（最多 FH_MAX_RETRY_ERRORS 次）
 *  - 上下文压缩：长时任务自动压缩早期消息（每 FH_CONTEXT_COMPACT_EVERY 次迭代）
 *  - 经验学习：会话完成后提取经验并保存
 *  - 模型性能追踪：自动记录各 provider 成功率，影响后续路由选择
 */
import type { ModelRouter } from '../models/model-router';
import type { ToolRegistry } from '../tools/tool.registry';
import type { ChatMessage, ChatResponse } from '../models/model.interface';
import type { CapabilityTag } from '../shared/types';
import type { EventLog } from '../runtime/event-log';
import type { ToolGuard } from '../tools/tool.interface';
import type { SandboxMode, SandboxRules } from '../tools/sandbox';
import type { HookConfig } from '../runtime/hooks';
import type { SessionStore } from '../runtime/session-store';
import type { SessionCheckpoint, SessionStatus } from '../runtime/session-persist';
import { SYSTEM_PROMPT } from './prompts';
import { planTask } from './planner';
```

```
30 import { buildRepoInstructionsPrompt, scopedInstructionsFor } from './repo-context';
31 import { discoverSkills, buildSkillIndexPrompt } from '../skills/skill-loader';
32 import { logger } from '../shared/logger';
33 import {
34   classifyError,
35   injectReflection,
36   countConsecutiveErrors,
37   logRecoveryAttempt,
38   type ErrorAnalysis,
39 } from './self-heal';
40 import { compactContext, shouldCompact, getCompactionThreshold } from './context-compactor';
41 import {
42   extractExperience,
43   upsertExperience,
44   retrieveRelevantExperiences,
45   generateExperiencePrompt,
46   updateExperienceUsage,
47   extractFixPattern,
48   type Experience,
49 } from './experience';
50
```

```
export interface OrchestratorSecurity {
  shellAllowlist: string[];
  requireApproval: boolean;
  /** P0-2：沙箱模式（缺省 workspace-write） */
  sandboxMode?: SandboxMode;
  /** P0-2：网络域名规则 */
  networkRules?: SandboxRules;
  /** P2-1：hooks 确定性控制 */
```

```
  hooks?: HookConfig[];
}

/**
 * P0-1 流式事件：编排器关键节点的实时事件流，CLI 可据此增量打印到 stdout。
 * 不引入任何渲染逻辑，事件与展示解耦（后续 TUI 可复用同一事件流）。
 */
export type OrchestratorEvent =
  | { type: 'model.response'; provider: string; model: string; content: string; toolCalls: string[] }
  | { type: 'tool.call'; name: string; args: Record<string, unknown> }
  | { type: 'tool.result'; name: string; ok: boolean; output: string }
  | { type: 'self-heal'; category: string; iteration: number }
  | { type: 'context.compact'; originalLength: number; compressedLength: number }
  | { type: 'session.end'; iterations: number; costUsd: number; ok: boolean };

export interface OrchestratorDeps {
  router: ModelRouter;
  tools: ToolRegistry;
  eventLog: EventLog;
  session: SessionStore;
  cwd: string;
```

```
  security: OrchestratorSecurity;
  approve?: (action: string) => Promise<boolean>;
  maxIterations?: number;
  /** M3：每轮迭代后落盘检查点（resume/diff/rollback 依赖） */
  persist?: (cp: SessionCheckpoint) => Promise<void>;
  /** M4：企业守卫（RBAC 策略 + 审批 + 审计），未注入时行为同 M3 */
  guard?: ToolGuard;
  /** M4：单任务成本上限（USD），超出即中止，0/undefined 表示不限 */
  maxCostUsd?: number;
  /** M6：错误自动重试上限（默认 3） */
  maxRetryErrors?: number;
  /** M6：上下文压缩触发阈值（默认 30 条消息） */
  contextCompactEvery?: number;
  /** M6：经验目录 */
  experienceDir?: string;
  /** P0-1：流式事件回调（实时输出进度，可选） */
  onEvent?: (ev: OrchestratorEvent) => void;
  /** P1-1：模型能力标签路由（缺省 ['code-gen']；子任务可传 ['code-gen','cheap'] 让低成本模型分担） */
  tags?: CapabilityTag[];
  /** P3-3：插件技能目录（合并进技能发现） */
  pluginSkillDirs?: string[];
```

```
  /** P9：外部中断信号（任务停止按钮），触发后中断编排循环与进行中的模型请求 */
  signal?: AbortSignal;
  /** P9：模型调用失败重试上限（默认 3 次，指数退避应对 429 限流） */
  maxModelRetries?: number;
}

/** M3 resume 上下文：携带已完成的对话与计数，避免重复执行 */
export interface ResumeContext {
```

```
109   messages: ChatMessage[];
110   iterations: number;
111   costUsd: number;
112   touchedFiles: string[];
113 }
114
115 export interface RunResult {
116   ok: boolean;
117   finalAnswer: string;
118   iterations: number;
119   costUsd: number;
120   logFile: string;
121   runId: string;
122   /** M6: 是否触发自愈循环 */
123   selfHealed?: boolean;
124   /** M6: 经验提取数量 */
125   experiencesExtracted?: number;
126 }
127
128 export class Orchestrator {
129   constructor(private readonly deps: OrchestratorDeps) {}
```

```
130
131  /** 成功工具调用结果缓存：key = toolName + JSON.stringify(args)，防止模型重复调用相同工具 */
132  private toolResultCache = new Map<string, { ok: boolean; output: string; error?: string }>();
133
134  async run(goal: string, resume?: ResumeContext): Promise<RunResult> {
135    const {
136      router, tools, eventLog, session, cwd, security, approve, persist, guard,
137      maxRetryErrors = 3,
138      contextCompactEvery,
139      experienceDir,
140    } = this.deps;
141    const maxIter = this.deps.maxIterations ?? 50;
142    const maxCost = this.deps.maxCostUsd ?? 0;
143    const compactThreshold = getCompactionThreshold({ compactEvery: contextCompactEvery });
144    // 每次 run 重置工具结果缓存
145    this.toolResultCache.clear();
146
147    let messages: ChatMessage[];
148    let loadedExperiences: Experience[] = [];
149    let baselineIterations = 0;
150    let carryCost = 0;
```

```
151     const touchedFiles: string[] = resume ? [...resume.touchedFiles] : [];
152     let selfHealed = false;
153     const errorHistory: ErrorAnalysis[] = [];
154
155     if (resume && resume.messages.length > 0) {
156       messages = resume.messages;
157       baselineIterations = resume.iterations;
158       carryCost = resume.costUsd;
```

```
159       await eventLog.append('session.resume', { fromIterations: baselineIterations });
160     } else {
161       // M6: 加权检索相关历史经验（强化学习召回）
162       loadedExperiences = experienceDir ? await retrieveRelevantExperiences(experienceDir, goal) : [];
163       const experiencePrompt = generateExperiencePrompt(loadedExperiences);
164       // P0-4: 仓库级 AGENTS.md 指令自动注入（存在时）
165       const repoPrompt = buildRepoInstructionsPrompt(cwd);
166       // P1-2: 技能索引渐进式披露（name+description 常驻，正文由 load_skill 按需加载）
167       const skillIndex = buildSkillIndexPrompt(discoverSkills(cwd, this.deps.pluginSkillDirs ?? []));
168       let systemPrompt = experiencePrompt ? `${SYSTEM_PROMPT}\n\n${experiencePrompt}` : SYSTEM_PROMPT;
169       if (repoPrompt) systemPrompt += repoPrompt;
170       if (skillIndex) systemPrompt += skillIndex;
171
172       const { messages: initMessages, plan } = planTask(goal);
173       messages = [{ role: 'system', content: systemPrompt }, ...initMessages];
174       session.append(messages[0]);
175       session.append(messages[1]);
176       await eventLog.append('plan', { steps: plan.steps });
177       await eventLog.append('session.start', { goal, cwd });
178     }
179
```

```
180     let finalAnswer = '';
181     let cost = carryCost;
182     let calls = 0;
183     let consecutiveErrors = 0;
184
185     // 检查点落盘（闭包引用最新 calls/cost/touchedFiles）
186     const emitCheckpoint = async (status: SessionStatus): Promise<void> => {
187       if (!persist) return;
188       const snap = session.snapshot();
189       await persist({
190         runId: snap.runId,
191         goal,
192         cwd,
193         createdAt: snap.createdAt,
194         updatedAt: new Date().toISOString(),
195         status,
196         iterations: baselineIterations + calls,
197         costUsd: cost,
198         messages: snap.messages,
199         touchedFiles,
200       });
```

```
201     };
202
203     if (!resume) await emitCheckpoint('running');
204
205     for (; calls < maxIter; calls++) {
206       // P9：任务停止——检查外部中断信号，触发即终止编排循环
207       if (this.deps.signal?.aborted) {
208         await emitCheckpoint('crashed');
```

```
209         return {
210           ok: false,
211           finalAnswer: '任务已被用户中断',
212           iterations: baselineIterations + calls,
213           costUsd: cost,
214           logFile: '',
215           runId: session.snapshot().runId,
216           selfHealed,
217         };
218       }
219       const startTime = Date.now();
220       // P9：模型调用失败轮询重试（应对 429 限流 / 瞬时网络抖动），最多 maxModelRetries 次，指数退避
221       let resp: ChatResponse | undefined;
222       let lastChatErr: unknown;
223       const maxModelRetries = this.deps.maxModelRetries ?? 3;
224       for (let attempt = 0; attempt <= maxModelRetries; attempt++) {
225         // 重试前再次检查中断信号
226         if (this.deps.signal?.aborted) {
227           await emitCheckpoint('crashed');
228           return {
229             ok: false,
```

```
230             finalAnswer: '任务已被用户中断',
231             iterations: baselineIterations + calls,
232             costUsd: cost,
233             logFile: '',
234             runId: session.snapshot().runId,
235             selfHealed,
236           };
237         }
238         try {
239           resp = await router.chat(
240             { messages, tools: tools.definitions(), temperature: 0, timeoutMs: 180000, signal: this.de
    ps.signal },
241             this.deps.tags ?? ['code-gen'],
242           );
243           break; // 成功，跳出重试
244         } catch (e) {
245           lastChatErr = e;
246           logger.warn('model call failed, will retry', {
247             attempt: attempt + 1,
248             maxRetries: maxModelRetries,
249             error: e instanceof Error ? e.message : String(e),
250           });
```

```
251           if (attempt < maxModelRetries) {
252             // 指数退避：1s, 2s, 4s...
253             await new Promise((r) => setTimeout(r, 1000 * Math.pow(2, attempt)));
254           }
255         }
256       }
257       if (!resp) {
258         throw lastChatErr instanceof Error ? lastChatErr : new Error('模型调用失败');
```

```
259       }
260       const latency = Date.now() - startTime;
261       cost += resp.costUsd;
262       const msg = resp.message;
263       messages.push(msg);
264       session.append(msg);
265
266       this.recordTouchedFiles(msg, touchedFiles);
267
268       await eventLog.append('model.response', {
269         provider: resp.providerId,
270         model: resp.model,
271         toolCalls: (msg.toolCalls ?? []).map((t) => t.name),
272         latencyMs: latency,
273       });
274       this.deps.onEvent?.({
275         type: 'model.response',
276         provider: resp.providerId,
277         model: resp.model,
278         content: msg.content || '',
279         toolCalls: (msg.toolCalls ?? []).map((t) => t.name),
```

```
      });
      await emitCheckpoint('running');

      // 无工具调用 → 视为任务完成
      if (!msg.toolCalls || msg.toolCalls.length === 0) {
        finalAnswer = msg.content;
        break;
      }

      // M4：单任务成本熔断（超预算立即停手，避免失控烧钱）
      if (maxCost > 0 && cost >= maxCost) {
        finalAnswer = `已达单任务成本上限 $${maxCost}（当前 $${cost.toFixed(6)}），任务中止。可调高角色策略 maxCostUsd 后用 resume 续跑。`;
        await eventLog.append('error', { reason: 'cost-limit-reached', costUsd: cost, maxCost });
        logger.warn('orchestrator hit cost limit', { runId: session.runId, cost, maxCost });
        calls++;
        break;
      }

      const roundErrors = await this.executeToolRound(msg, { tools, eventLog, session, cwd, security, approve, guard }, messages);
```

```
300       // M6: 错误检测与自愈循环
301       if (roundErrors > 0) {
302         const rec = await this.handleRecovery(msg, {
303           iteration: calls,
304           consecutiveErrors,
305           maxRetryErrors,
306           eventLog,
307           goal,
308           errorHistory,
```

```
309           messages,
310           sessionRunId: session.runId,
311         });
312         if (rec.signal === 'break') {
313           finalAnswer = rec.finalAnswer;
314           calls++;
315           break;
316         }
317         if (rec.signal === 'continue') {
318           messages = rec.messages;
319           consecutiveErrors = rec.consecutiveErrors;
320           selfHealed = rec.selfHealed;
321           continue; // 跳过本轮计数，直接进入下一轮
322         }
323       } else {
324         // 本轮无错误，重置连续错误计数
325         consecutiveErrors = 0;
326       }
327
328       await emitCheckpoint('running');
329
```

```
330       // M6: 上下文压缩
331       if (shouldCompact(messages, compactThreshold)) {
332         const { messages: compacted, stats } = compactContext(messages, 10);
333         messages = compacted;
334         await eventLog.append('context.compact', { ...stats } as Record<string, unknown>);
335         this.deps.onEvent?.({
336           type: 'context.compact',
337           originalLength: stats.originalLength,
338           compressedLength: stats.compressedLength,
339         });
340         logger.info('context compaction applied', {
341           originalLength: stats.originalLength,
342           compressedLength: stats.compressedLength,
343         });
344       }
345     }
346
347     if (baselineIterations + calls >= maxIter && !finalAnswer) {
348       const hint = touchedFiles.length > 0
349         ? `可在对话区继续发送"继续"二字，让任务以 resume 方式接着跑（已生成/修改的文件位于工作区）。`
350         : `可点击「继续」按钮让任务接着执行，或在对话区细化任务目标重新提交。`;
```

```
351       finalAnswer = `已达到最大迭代次数 ${maxIter} 轮，任务可能未完全完成。${hint}`;
352       await eventLog.append('error', { reason: 'max-iterations-reached', maxIter, touchedFiles });
353       logger.warn('orchestrator reached max iterations', { runId: session.runId, maxIter });
354     }
355
356     // M6: 提取经验并以 upsert 强化写入（同一模式多次验证会累积权重，而非无限追加）
357     let experiencesExtracted = 0;
358     if (experienceDir && finalAnswer) {
```

```
359       const experiences = extractExperience(messages, session.runId);
360       for (const exp of experiences) {
361         await upsertExperience(experienceDir, exp);
362       }
363       // 自愈成功：额外固化一条「修复经验」，强化后续同类错误的闭环
364       if (selfHealed) {
365         const fixExp = extractFixPattern(messages);
366         if (fixExp) {
367           await upsertExperience(experienceDir, fixExp);
368           experiences.push(fixExp);
369         }
370       }
371       // 更新「被加载并用于本次任务」的经验的统计（强化其权重）
372       for (const exp of loadedExperiences) {
373         await updateExperienceUsage(experienceDir, exp.id);
374       }
375       experiencesExtracted = experiences.length;
376       await eventLog.append('experience.extracted', { count: experiencesExtracted, selfHealed });
377     }
378
379     await emitCheckpoint('done');
```

```
380     await eventLog.append('session.end', {
381       iterations: baselineIterations + calls,
382       costUsd: cost,
383       selfHealed,
384       experiencesExtracted,
385     });
386     this.deps.onEvent?.({
387       type: 'session.end',
388       iterations: baselineIterations + calls,
389       costUsd: cost,
390       ok: finalAnswer.length > 0,
391     });
392
393     return {
394       ok: finalAnswer.length > 0,
395       finalAnswer,
396       iterations: baselineIterations + calls,
397       costUsd: cost,
398       logFile: eventLog.filePath,
399       runId: session.runId,
400       selfHealed,
```

```
401       experiencesExtracted,
402     };
403   }
404
405   /** 记录被 write/edit 类工具改动的文件路径（用于检查点与 diff） */
406   private recordTouchedFiles(msg: ChatMessage, touchedFiles: string[]): void {
407     for (const tc of msg.toolCalls ?? []) {
408       const p = (tc.arguments as { path?: unknown } | undefined)?.path;
```

```
409       if (
410         (tc.name === 'write_file' || tc.name === 'edit_file') &&
411         typeof p === 'string' &&
412         !touchedFiles.includes(p)
413       ) {
414         touchedFiles.push(p);
415       }
416     }
417   }
418
419   /** 执行本轮所有工具调用，将结果以 tool 消息回填，返回本轮失败次数 */
420   private async executeToolRound(
421     msg: ChatMessage,
422     ctx: {
423       tools: ToolRegistry;
424       eventLog: EventLog;
425       session: SessionStore;
426       cwd: string;
427       security: OrchestratorSecurity;
428       approve?: (action: string) => Promise<boolean>;
429       guard?: ToolGuard;
```

```
430     },
431     messages: ChatMessage[],
432   ): Promise<number> {
433     let roundErrors = 0;
434     for (const tc of msg.toolCalls ?? []) {
435       // P2-3：JIT 注入路径级规则——工具操作涉及的文件命中 AGENTS.md 的
436       // paths frontmatter 时，把对应规则作为 system 消息注入（按文件去重）
437       const fileArg = (tc.arguments as { path?: unknown } | undefined)?.path;
438       if (typeof fileArg === 'string') {
439         const scoped = scopedInstructionsFor(ctx.cwd, fileArg);
440         if (scoped && !messages.some((m) => m.role === 'system' && m.content.includes(`匹配 ${fileArg}
    `))) {
441           const ruleMsg: ChatMessage = { role: 'system', content: scoped };
442           messages.push(ruleMsg);
443           ctx.session.append(ruleMsg);
444         }
445       }
446       await ctx.eventLog.append('tool.call', { name: tc.name, args: tc.arguments });
447       this.deps.onEvent?.({ type: 'tool.call', name: tc.name, args: tc.arguments });
448
449       // 去重检测：相同工具+相同参数如果之前已成功，直接返回缓存结果并警告模型不要重复调用
450       const cacheKey = tc.name + '::' + JSON.stringify(tc.arguments ?? {});
```

```
451       const cached = this.toolResultCache.get(cacheKey);
452       let result: { ok: boolean; output: string; error?: string };
453       if (cached && cached.ok) {
454         const warning = `[重复调用警告] 工具 ${tc.name} 用相同参数之前已成功执行过，以下是缓存的结果。请不要重复调用相同工具，直接利用已有信息推进任务。\n`;
455         result = { ok: true, output: warning + cached.output };
456         await ctx.eventLog.append('tool.result', { name: tc.name, ok: true, output: result.output.slice(0, 500) });
457         this.deps.onEvent?.({ type: 'tool.result', name: tc.name, ok: true, output: result.output.slice(0, 500) });
458       } else {
```

```
459         result = await ctx.tools.execute(tc.name, tc.arguments, {
460           runId: ctx.session.runId,
461           cwd: ctx.cwd,
462           security: ctx.security,
463           approve: ctx.approve,
464           guard: ctx.guard,
465         });
466         // 成功结果写入缓存（失败的不缓存，允许模型重试修复）
467         if (result.ok) {
468           this.toolResultCache.set(cacheKey, { ok: result.ok, output: result.output, error: result.err
    or });
469         }
470         await ctx.eventLog.append('tool.result', {
471           name: tc.name,
472           ok: result.ok,
473           output: result.output.slice(0, 500),
474         });
475         this.deps.onEvent?.({ type: 'tool.result', name: tc.name, ok: result.ok, output: result.outpu
    t.slice(0, 500) });
476       }
477       const content = result.ok ? result.output : `错误: ${result.error}`;
478       const toolMsg: ChatMessage = { role: 'tool', content, toolCallId: tc.id };
```

```
      messages.push(toolMsg);
      ctx.session.append(toolMsg);
      if (!result.ok) roundErrors++;
    }
    return roundErrors;
  }

  /** 错误检测与自愈：返回 'break'（达重试上限）/ 'continue'（已注入反思）/ 'none'（未触发） */
  private async handleRecovery(
    _msg: ChatMessage,
    input: {
      iteration: number;
      consecutiveErrors: number;
      maxRetryErrors: number;
      eventLog: EventLog;
      goal: string;
      errorHistory: ErrorAnalysis[];
      messages: ChatMessage[];
      sessionRunId: string;
    },
  ): Promise<{ signal: 'break' | 'continue' | 'none'; messages: ChatMessage[]; consecutiveErrors: numb
er; selfHealed: boolean; finalAnswer: string }> {
```

```
500     const { messages } = input;
501     const { failed, errors } = countConsecutiveErrors(messages, input.maxRetryErrors);
502     if (errors <= input.consecutiveErrors) {
503       return { signal: 'none', messages, consecutiveErrors: input.consecutiveErrors, selfHealed: fals
    e, finalAnswer: '' };
504     }
505     const lastToolMsg = messages[messages.length - 1];
506     // classifyError 现在对未匹配错误返回 'unknown' 兜底，永不返回 null，
507     // 确保达到 maxRetryErrors 时 failed 分支一定能 break，不会被绕过。
508     const errorAnalysis = classifyError(lastToolMsg.content || '', '');
```

```
509     // 从消息历史中提取最后一次工具调用（工具名+参数），用于生成更精准的反思提示
510     let lastToolCall: { name: string; args: Record<string, unknown> } | undefined;
511     for (let i = messages.length - 2; i >= 0; i--) {
512       const m = messages[i];
513       if (m.role === 'assistant' && m.toolCalls && m.toolCalls.length > 0) {
514         const tc = m.toolCalls[m.toolCalls.length - 1];
515         lastToolCall = { name: tc.name, args: tc.arguments ?? {} };
516         break;
517       }
518     }
519     input.errorHistory.push(errorAnalysis);
520     await logRecoveryAttempt(input.eventLog, input.iteration, errorAnalysis, false);
521     const consecutiveErrors = errors;
522     if (failed) {
523       const finalAnswer = `任务执行遇到连续 ${input.maxRetryErrors} 次错误，已尝试自动修复但未能成功。错误类型: ${input.errorHistory.map((e) => e.category).join(', ')}。请检查工作区与日志，或使用 resume 续跑。`;
524       await input.eventLog.append('error', { reason: 'max-retry-errors', errors: input.errorHistory });
525       logger.warn('orchestrator hit max retry errors', { runId: input.sessionRunId, errors: input.errorHistory });
526       return { signal: 'break', messages, consecutiveErrors, selfHealed: false, finalAnswer };
527     }
528     const selfHealed = true;
```

```
529     const newMessages = injectReflection(messages, errorAnalysis, input.goal, lastToolCall);
530     await input.eventLog.append('self-heal', { category: errorAnalysis.category, iteration: input.iteration });
531     this.deps.onEvent?.({ type: 'self-heal', category: errorAnalysis.category, iteration: input.iteration });
532     logger.info('self-heal: injected reflection', { iteration: input.iteration, category: errorAnalysis.category });
533     return { signal: 'continue', messages: newMessages, consecutiveErrors, selfHealed, finalAnswer: '' };
534   }
535 }
536
```

// ===== src/agent/planner.ts =====

```
1 /**
2  * 飞虹 Code (Muse Code 参照复刻)
3  * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
4  *
5  * 规划器：
6  *   - planTask(goal)：M1 最小版，返回可读步骤（单代理执行用）。
7  *   - decomposeGoal(goal)：M2 升级，把大目标拆成可并行的子任务列表（多子代理用）。
8  *     离线用规则拆分；联调时可由模型生成更智能的分解。
9  */
```

```ts
import type { ChatMessage } from '../models/model.interface';

export interface Plan {
  steps: string[];
}

export interface SubTask {
  id: string;
  title: string;
  /** 派发给子代理的自然语言目标 */
```

```
  goal: string;
}

export function planTask(goal: string): { messages: ChatMessage[]; plan: Plan } {
  const plan: Plan = {
    steps: [
      '勘察工作区：了解目录结构与相关文件',
      '制定并应用变更（read/write/edit/shell）',
      '运行测试与构建验证变更有效',
      '收尾并给出总结',
    ],
  };
  const messages: ChatMessage[] = [{ role: 'user', content: goal }];
  return { messages, plan };
}

/**
 * 把目标拆成子任务。
 * 规则拆分（离线/无模型时）：
 *  - 含显式分隔（换行、分号、句号 + 序号、'并且'/'同时'/'分别'/'一方面…另一方面'）则按片段拆。
 *  - 否则整体作为一个子任务。
```

```
 * 生成的子任务目标彼此独立，适合并行隔离执行。
 */
export function decomposeGoal(goal: string): SubTask[] {
  const normalized = goal.trim();
  if (!normalized) return [];

  const fragments = splitByConjunctions(normalized);
  if (fragments.length <= 1) {
    return [{ id: 't1', title: '主线任务', goal: normalized }];
  }

  return fragments.map((frag, i) => ({
    id: `t${i + 1}`,
    title: frag.length > 24 ? frag.slice(0, 24) + '…' : frag,
    goal: frag,
  }));
}

/** 按中文/英文常见并列连词与标点拆分。
 *  拆分后会合并过短的片段（< MIN_FRAGMENT_LEN 字）回前一个，避免连贯句子被撕碎。 */
const MIN_FRAGMENT_LEN = 12;
```

```
62
63 function splitByConjunctions(text: string): string[] {
64   const raw = text
65     .split(/\n+/)
66     .flatMap((line) => line.split(/(?:，|；|。|并且|同时|分别|以及|还有|另外|一方面|另一方面)/))
67     .map((s) => s.trim())
68     .filter(Boolean);
69   // 合并过短片段：长度 < MIN_FRAGMENT_LEN 的片段合并回前一个，避免"先读文件"被拆成独立子任务
```

```
70  const merged: string[] = [];
71  for (const frag of raw) {
72    if (merged.length > 0 && frag.length < MIN_FRAGMENT_LEN) {
73      merged[merged.length - 1] += ', ' + frag;
74    } else {
75      merged.push(frag);
76    }
77  }
78  // 若合并后只有一段，尝试按「做X和Y」式并列再拆
79  if (merged.length <= 1) {
80    const alt = text.split(/(?:和|与|、|以及|并|加上)/).map((s) => s.trim()).filter(Boolean);
81    if (alt.length > 1) {
82      const mergedAlt: string[] = [];
83      for (const frag of alt) {
84        if (mergedAlt.length > 0 && frag.length < MIN_FRAGMENT_LEN) {
85          mergedAlt[mergedAlt.length - 1] += '和' + frag;
86        } else {
87          mergedAlt.push(frag);
88        }
89      }
90      if (mergedAlt.length > 1) return mergedAlt;
```

```
    }
  }
  return merged;
}

```

// ===== src/agent/self-heal.ts =====

```
/**
 * 飞虹 Code (Muse Code 参照复刻)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 自我修复循环（Self-Healing Loop）：
 * - 错误模式分类（编译失败/运行时错误/路径穿越/超时等）
 * - 反思提示词生成（基于错误提出修复建议）
 * - 自动重试机制（最多 N 次）
 */
import type { ChatMessage } from '../models/model.interface';
import type { EventLog } from '../runtime/event-log';
import { logger } from '../shared/logger';

```

```
14 export type ErrorCategory =
15   | 'compile-error'
16   | 'runtime-error'
17   | 'path-traversal'
18   | 'timeout'
19   | 'permission-denied'
20   | 'model-error'
21   | 'file-not-found'
```

```
  | 'build-error'
  | 'command-not-found'
  | 'invalid-args'
  | 'unknown';

export interface ErrorAnalysis {
  category: ErrorCategory;
  message: string;
  fixHint: string;
}

interface ErrorRule {
  category: ErrorCategory;
  keywords: string[];
  fixHint: string;
}

/** 错误分类规则表（顺序即优先级，命中即返回） */
const ERROR_RULES: ErrorRule[] = [
  {
    category: 'compile-error',
```

```
43      keywords: ['error ts', 'typescript error', 'cannot find module', 'syntax error', '编译', 'tsc'],
44      fixHint: '检查 TypeScript 类型错误、缺失依赖或路径拼写错误。尝试运行 npm run build 查看完整错误信息。',
45    },
46    {
47      category: 'runtime-error',
48      keywords: ['undefined is not', 'cannot read property', 'referenceerror', 'typeerror', '未定义', 'is not a function'],
49      fixHint: '检查变量是否已定义、函数调用是否正确、空值处理是否完善。',
50    },
51    {
52      category: 'path-traversal',
53      keywords: ['path traversal', 'outside workspace', 'safe-path', 'forbidden', '路径', '穿越'],
54      fixHint: '文件路径包含非法字符或试图访问工作区外部。使用相对路径，避免 ../ 穿越。',
55    },
56    {
57      category: 'timeout',
58      keywords: ['timeout', 'timed out', 'ETIMEDOUT', '超时'],
59      fixHint: '命令执行超时。检查是否存在死循环或需要长时间运行的进程。',
60    },
61    {
62      category: 'permission-denied',
63      keywords: ['permission denied', 'eacces', 'eperm', '权限', '拒绝'],
```

```
64      fixHint: '文件权限不足或路径不存在。检查文件权限和目录结构。',
65    },
66    {
67      category: 'file-not-found',
68      keywords: ['enoent', 'no such file', '读取失败', '文件不存在', 'not found', 'eisdir', '不存在'],
69      fixHint: '目标文件或目录不存在。先用 list_dir 勘察当前目录结构，确认正确路径后再操作；不要反复尝试同一不存在的路径。',
70    },
71    {
```

```
72      category: 'build-error',
73      keywords: ['missing script', 'npm error', 'build failed', '编译失败', '构建失败', 'exit code 1', 'command failed'],
74      fixHint: '构建命令执行失败。检查 package.json 是否存在对应 script、工作目录是否正确、依赖是否已安装。先用 list_dir 确认目录结构，再决定用什么构建命令。',
75    },
76    {
77      category: 'command-not-found',
78      keywords: ['command not found', 'not recognized', 'is not recognized', '找不到命令', '命令不存在', 'enoent command'],
79      fixHint: '命令不存在或未安装。检查命令拼写是否正确、是否需要先安装依赖、是否在正确的工作目录下执行。',
80    },
81    {
82      category: 'invalid-args',
83      keywords: ['参数校验失败', '参数警告', 'unknown tool', '未知工具', 'invalid argument', 'unexpected argument'],
84      fixHint: '工具参数错误。检查是否把 A 工具的参数传给了 B 工具，每个工具只接受自己文档中定义的参数。先确认工具名称，再传对应参数。',
85    },
86    {
87      category: 'model-error',
88      keywords: ['api error', 'rate limit', '429', '500', '模型', '限流'],
89      fixHint: '模型 API 错误。检查 API 密钥、速率限制或网络连通性。',
90    },
```

```
];

/** 错误分类器：根据工具返回结果判断错误类型（规则表驱动，中英双语识别）。
 *  未命中任何规则时返回 'unknown' 兜底分类，确保 orchestrator 的重试上限与自愈逻辑不会被绕过。 */
export function classifyError(output: string, error?: string): ErrorAnalysis {
  const text = (error || output).toLowerCase();
  for (const rule of ERROR_RULES) {
    if (rule.keywords.some((kw) => text.includes(kw))) {
      return { category: rule.category, message: error || output, fixHint: rule.fixHint };
    }
  }
  return {
    category: 'unknown',
    message: error || output,
    fixHint: '未识别的错误类型。请仔细阅读错误信息，调整执行策略，避免重复相同操作。',
  };
}

/** 生成反思提示词，指导模型基于错误进行修复。
 *  包含具体工具名、参数、错误详情，避免笼统的"请反思"导致模型重复同样错误。 */
export function generateReflectPrompt(errorAnalysis: ErrorAnalysis, _goal?: string, lastToolCall?: { name: string; args: Record<string, unknown> }): string {
```

```
  const toolInfo = lastToolCall
    ? `\n**出错工具**: ${lastToolCall.name}\n**传入参数**: ${JSON.stringify(lastToolCall.args)}\n`
    : '';
  return `⚠️ 上一轮工具执行失败，请仔细反思并修正策略：

**错误类型**: ${errorAnalysis.category}
**错误详情**: ${errorAnalysis.message}${toolInfo}
**修复建议**: ${errorAnalysis.fixHint}

请严格遵守以下规则：
```

```
1. 不要再用相同参数重复调用同一个失败的工具——这会继续失败
2. 检查工具名称是否正确、参数是否属于该工具（每个工具只接受自己文档中定义的参数）
3. 如有必要，先用 list_dir 勘察目录结构，或用 read_file 确认文件存在
4. 如果某个工具连续失败，换一种方式或换一个工具实现目标
5. 修正后再调用工具，不要盲目重试
`;
}

/** 将反思消息注入对话上下文 */
export function injectReflection(messages: ChatMessage[], errorAnalysis: ErrorAnalysis, goal: string, lastToolCall?: { name: string; args: Record<string, unknown> }): ChatMessage[] {
  const reflectMsg: ChatMessage = {
    role: 'user',
    content: generateReflectPrompt(errorAnalysis, goal, lastToolCall),
  };
  return [...messages, reflectMsg];
}

/** 统计连续失败次数 */
export function countConsecutiveErrors(messages: ChatMessage[], maxRetries: number): { failed: boolean; errors: number } {
  let errors = 0;
```

```
142   for (let i = messages.length - 1; i >= 0; i--) {
143     const msg = messages[i];
144     if (msg.role === 'tool' && msg.content.includes('错误:')) {
145       errors++;
146     } else if (msg.role === 'tool' && !msg.content.includes('错误:')) {
147       // 工具成功执行，重置计数
148       break;
149     } else if (msg.role === 'assistant' && errors > 0) {
150       // 助手在连续错误后仍无工具调用，说明已尝试修复但失败
151       break;
152     }
153   }
154   return {
155     failed: errors >= maxRetries,
156     errors,
157   };
158 }
159
160 /** 构建反思上下文（包含历史错误） */
161 export function buildReflectContext(messages: ChatMessage[], errors: ErrorAnalysis[]): {
162   messages: ChatMessage[];
```

```
  contextLength: number;
} {
  // 保留最近 5 轮完整对话
  const recentMessages = messages.slice(-10);

  // 将错误历史作为系统提示注入
  const errorHistory = errors.map((e, i) =>
    `[第 ${i + 1} 次失败] 类型: ${e.category}, 详情: ${e.message.slice(0, 200)}`,
  ).join('\n');
```

```
172
173   const systemHint: ChatMessage = {
174     role: 'system',
175     content: `⚠️ 连续失败记录（自动学习模式）\n\n${errorHistory}\n\n请从错误中学习，调整策略，避免重复失败。`,
176   };
177
178   return {
179     messages: [systemHint, ...recentMessages],
180     contextLength: messages.length,
181   };
182 }
183
184 /** 记录修复事件到日志 */
185 export async function logRecoveryAttempt(
186   eventLog: Pick<EventLog, 'append'>,
187   iteration: number,
188   errorAnalysis: ErrorAnalysis,
189   success: boolean,
190 ): Promise<void> {
191   await eventLog.append('self-heal.attempt', {
```

```
    iteration,
    category: errorAnalysis.category,
    message: errorAnalysis.message.slice(0, 500),
    success,
    timestamp: new Date().toISOString(),
  });
  logger.info('self-heal recovery attempt', { iteration, category: errorAnalysis.category, success });
}

```

// ===== src/agent/code-writer.ts =====

```
/**
 * 飞虹 Code (Muse Code 参照复刻)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 自主代码编写器（M8）：
 * - 任务规划 → 代码编写 → 测试生成 → 审查反馈 → 自愈修复 → 迭代优化
 * - 内置代码生成模板（API 路由、Model、工具骨架）
 * - 自动测试生成（Jest/Vitest）
 * - 质量门禁集成（审查规则 + 分析结果）
 * - 安全修复：仅对已知安全模式做针对性修复，杜绝「通配正则整体覆盖文件」的破坏性修复
```

```
 * - 自愈闭环：审查 → 修复 → 复审查，多轮收敛至无高优先级问题
 */
import { existsSync, mkdirSync, writeFileSync, readFileSync } from 'fs';
import { join, dirname } from 'path';
import type { ReviewRule, ReviewIssue } from './code-review';
import { reviewCode } from './code-review';
import { analyzeFile } from '../tools/analysis/code-analyzer';
import { generateJestTest, inferTestCases } from '../tools/generator/test-generator';
```

```
import {
  generateApiRoute,
  generateModel,
  generateToolTemplate,
} from '../tools/generator/code-generator';

export interface CodeWriterStep {
  type: 'plan' | 'write' | 'test' | 'review' | 'fix' | 'summary';
  content: string;
  file?: string;
}

export interface CodeWriterResult {
  success: boolean;
  steps: CodeWriterStep[];
  finalFiles: string[];
  issuesFound: number;
  issuesFixed: number;
  summary: string;
}

```

```
/** 安全修复规则：仅返回已知安全的针对性修复，无匹配返回 null（不做破坏性通配替换） */
interface SafeFix {
  pattern: RegExp;
  replacement: string;
}

/** 自主编写器状态机 */
export class CodeWriter {
  private steps: CodeWriterStep[] = [];
  private filesCreated: string[] = [];
  private issuesFound = 0;
  private issuesFixed = 0;
  private readonly rules: ReviewRule[];
  private readonly cwd: string;

  constructor(cwd: string, rules: ReviewRule[] = []) {
    this.cwd = cwd;
    this.rules = rules;
  }

  /** 阶段 1：规划 */
```

```
61  plan(goal: string): CodeWriterStep {
62    const step: CodeWriterStep = {
63      type: 'plan',
64      content: this.buildPlan(goal),
65    };
66    this.steps.push(step);
67    console.log(`[M8 Plan] ${step.content}`);
68    return step;
```

```
  }

  /** 阶段 2：编写代码 */
  write(code: string, filePath: string): CodeWriterStep {
    const absPath = join(this.cwd, filePath);
    mkdirSync(dirname(absPath), { recursive: true });
    writeFileSync(absPath, code, 'utf8');
    this.filesCreated.push(filePath);
    const step: CodeWriterStep = {
      type: 'write',
      content: `已生成文件: ${filePath} (${code.length} 字符)`,
      file: filePath,
    };
    this.steps.push(step);
    console.log(`[M8 Write] ${step.content}`);
    return step;
  }

  /** 阶段 2b：使用模板生成代码 */
  writeTemplate(
    template: 'api-route' | 'model' | 'tool',
```

```
90     params: Record<string, string>,
91   ): CodeWriterStep {
92     let code: string;
93     let filename: string;
94     const result =
95       template === 'api-route'
96         ? generateApiRoute(params.method, params.path, params.controller)
97         : template === 'model'
98           ? generateModel(params.name, JSON.parse(params.fields || '{}'))
99           : generateToolTemplate(params.name);
100    if (!result.success || !result.content) {
101      return { type: 'write', content: `模板生成失败: ${result.error || '未知错误'}` };
102    }
103    code = result.content;
104    if (template === 'api-route') {
105      // Windows 文件名不含 ':'，将路径段中非法字符替换为 '-'
106      filename = params.path.replace(/^\//, '').replace(/[^a-zA-Z0-9-]/g, '-').replace(/-+/g, '-').rep
lace(/-$/, '') + '.ts';
107    } else if (template === 'model') {
108      filename = params.name + '.ts';
109    } else {
110      filename = params.name + '.tool.ts';
```

```
111     }
112     return this.write(code, join('generated', filename));
113   }
114
115   /** 阶段 3：生成测试 */
116   test(targetFile: string, functionName?: string): CodeWriterStep {
117     const absPath = join(this.cwd, targetFile);
118     if (!existsSync(absPath)) {
```

```
119       return { type: 'test', content: `跳过测试生成：文件不存在 ${targetFile}` };
120     }
121     const content = readFileSync(absPath, 'utf8');
122     const funcName = functionName || this.inferFunctionName(content);
123     const testCases = inferTestCases(funcName, content.slice(0, 500));
124     const result = generateJestTest(targetFile, funcName, testCases);
125     const testFile = targetFile.replace('.ts', '.test.ts');
126     return this.write(result.content, testFile);
127   }
128
129   /** 阶段 4：审查 */
130   review(filePath: string): CodeWriterStep {
131     const absPath = join(this.cwd, filePath);
132     if (!existsSync(absPath)) {
133       return { type: 'review', content: `跳过审查：文件不存在 ${filePath}` };
134     }
135     const content = readFileSync(absPath, 'utf8');
136     const result = reviewCode(filePath, content, this.rules.length ? this.rules : undefined);
137     this.issuesFound += result.issues.length;
138     const step: CodeWriterStep = {
139       type: 'review',
```

```
140       content: this.formatReviewResult(result),
141       file: filePath,
142     };
143     this.steps.push(step);
144     console.log(`[M8 Review] ${step.content}`);
145     return step;
146   }
147
148   /**
149    * 阶段 5：安全修复高优先级问题
150    * 关键安全约束：仅对「硬编码密钥 / SQL 拼接」等已知安全模式做针对性替换（替换首个匹配），
151    * 任何无明确安全方案的问题一律跳过并提示人工处理，绝不使用通配正则把整个文件覆盖为建议文本。
152    */
153   fix(filePath: string, maxFixes = 3): CodeWriterStep {
154     const absPath = join(this.cwd, filePath);
155     if (!existsSync(absPath)) {
156       return { type: 'fix', content: `文件不存在 ${filePath}` };
157     }
158     const content = readFileSync(absPath, 'utf8');
159     const result = reviewCode(filePath, content, this.rules.length ? this.rules : undefined);
160     const highIssues = result.issues.filter((i) => i.severity === 'high').slice(0, maxFixes);
```

```
161     if (highIssues.length === 0) {
162       return { type: 'fix', content: '无高优先级问题需要修复' };
163     }
164
165     let newContent = content;
166     const applied: string[] = [];
167     for (const issue of highIssues) {
168       const safeFix = this.issueToSafeFix(issue);
```

```
169       if (!safeFix) continue; // 仅做安全修复
170       const next = newContent.replace(safeFix.pattern, safeFix.replacement);
171       if (next !== newContent) {
172         newContent = next;
173         applied.push(issue.message);
174       }
175     }
176
177     if (applied.length === 0) {
178       return {
179         type: 'fix',
180         content: `高优先级问题无安全自动修复，建议人工处理: ${highIssues.map((i) => i.message).join(', ')}
    `,
181         file: filePath,
182       };
183     }
184
185     writeFileSync(absPath, newContent, 'utf8');
186     this.issuesFixed += applied.length;
187     return {
188       type: 'fix',
```

```
      content: `已安全修复 ${applied.length} 个高优先级问题: ${applied.join(', ')}`,
      file: filePath,
    };
  }

  /**
   * 阶段 5b：自愈闭环
   * 审查 → 修复 → 复审查，最多 maxRounds 轮，直到无高优先级问题或达上限。
   * 修复动作复用安全修复（不会破坏文件），输出每轮收敛情况。
   */
  selfHealFix(filePath: string, maxRounds = 3): CodeWriterStep[] {
    const steps: CodeWriterStep[] = [];
    for (let round = 0; round < maxRounds; round++) {
      this.review(filePath); // 推入审查步骤
      const fixStep = this.fix(filePath);
      steps.push(fixStep);

      // 静默复审查（不重复推步骤），判定是否收敛
      const absPath = join(this.cwd, filePath);
      if (!existsSync(absPath)) break;
      const content = readFileSync(absPath, 'utf8');
      const re = reviewCode(filePath, content, this.rules);
```

```
    const high = re.issues.filter((i) => i.severity === 'high').length;
    if (high === 0) {
      steps.push({ type: 'fix', content: `自愈收敛：第 ${round + 1} 轮后无高优先级问题` });
      return steps;
    }
  }
  steps.push({ type: 'fix', content: `已达自愈上限(${maxRounds}轮)，剩余高优先级问题建议人工复核` });
  return steps;
```

```
219   }
220
221   /** 阶段 6：分析代码质量 */
222   analyze(filePath: string): CodeWriterStep {
223     const absPath = join(this.cwd, filePath);
224     if (!existsSync(absPath)) {
225       return { type: 'review', content: `文件不存在 ${filePath}` };
226     }
227     const content = readFileSync(absPath, 'utf8');
228     const analysis = analyzeFile(filePath, content);
229     return {
230       type: 'review',
231       content: this.formatAnalysisResult(analysis),
232       file: filePath,
233     };
234   }
235
236   /** 总结 */
237   summary(): CodeWriterStep {
238     const step: CodeWriterStep = {
239       type: 'summary',
```

```
240       content: this.buildSummary(),
241     };
242     this.steps.push(step);
243     return step;
244   }
245
246   /** 完整流程：规划 → 编写 → 测试 → 审查 → 自愈修复 → 总结 */
247   async run(goal: string, code: string, filePath: string): Promise<CodeWriterResult> {
248     this.plan(goal);
249     this.write(code, filePath);
250     this.test(filePath);
251     this.review(filePath);
252     this.analyze(filePath);
253     this.selfHealFix(filePath, 3);
254     this.summary();
255     return this.getResult();
256   }
257
258   /** 批量编写多个文件 */
259   async runBatch(
260     tasks: Array<{ goal: string; code: string; filePath: string }>,
```

```
261   ): Promise<CodeWriterResult> {
262     for (const task of tasks) {
263       await this.run(task.goal, task.code, task.filePath);
264     }
265     return this.getResult();
266   }
267
268   private getResult(): CodeWriterResult {
```

```
269     return {
270       success: this.issuesFound - this.issuesFixed === 0,
271       steps: this.steps,
272       finalFiles: this.filesCreated,
273       issuesFound: this.issuesFound,
274       issuesFixed: this.issuesFixed,
275       summary: this.buildSummary(),
276     };
277   }
278
279   private buildPlan(goal: string): string {
280     return `任务规划：${goal}\n预期输出文件：待编写\n策略：先规划结构，再逐步实现`;
281   }
282
283   private formatReviewResult(result: {
284     file: string;
285     issues: { severity: string; message: string }[];
286     passed: boolean;
287   }): string {
288     const high = result.issues.filter((i) => i.severity === 'high').length;
289     const med = result.issues.filter((i) => i.severity === 'medium').length;
```

```
290     const low = result.issues.filter((i) => i.severity === 'low').length;
291     return `${result.file}: ${result.issues.length} 个问题 (高:${high} 中:${med} 低:${low}) ${result.passed ? '✅ 通过' : '❌ 未通过'}`;
292   }
293
294   private formatAnalysisResult(analysis: {
295     issues: { type: string; severity: string; message: string }[];
296     metrics: { complexity: number };
297   }): string {
298     const highIssues = analysis.issues.filter((i) => i.severity === 'high').length;
299     return `代码质量分析: 复杂度=${analysis.metrics.complexity} 高优问题=${highIssues} 总问题=${analysis.issues.length}`;
300   }
301
302   private buildSummary(): string {
303     return `M8 自主编写完成: ${this.filesCreated.length} 文件, ${this.issuesFound} 问题发现, ${this.issuesFixed} 问题修复`;
304   }
305
306   private inferFunctionName(content: string): string {
307     const match = content.match(/export\s+(?:async\s+)?function\s+(\w+)/);
308     return match?.[1] || 'main';
309   }
```

```
310
311  /**
312   * 将审查问题映射到「安全修复」。
313   * 仅对硬编码密钥、SQL 拼接两类有明确安全方案的问题生成针对性正则；其余一律返回 null。
314   */
315  private issueToSafeFix(issue: ReviewIssue): SafeFix | null {
316    if (/硬编码|密码|密钥|secret|token|api[_-]?key/i.test(issue.message)) {
317      // 匹配变量名含敏感关键词（password/secret/token/apiKey）且赋值为字符串字面量的模式
318      // 例如：const dbPassword = "password-abc-123";  或  const apiKey = 'sk-123';
```

```
319       return {
320         pattern: /(\w*[Pp]assword\w*|\w*[Ss]ecret\w*|\w*[Tt]oken\w*|\w*[Aa][Pp][Ii]?\w*[Kk]ey\w*)\s*=\s*(['"`])([^'"`]+)\2/gi,
321         replacement: '$1 = process.env.FH_SECRET',
322       };
323     }
324     if (/sql/i.test(issue.message)) {
325       return {
326         pattern: /(['"`])[\s\S]*?select[\s\S]*?\1/gi,
327         replacement: '/* 使用参数化查询，避免 SQL 拼接 */',
328       };
329     }
330     return null;
331   }
332 }
333
334 /** 便捷函数 */
335 export function createCodeWriter(cwd: string, rules?: ReviewRule[]): CodeWriter {
336   return new CodeWriter(cwd, rules);
337 }
338
```

```
// ===== src/agent/experience.ts =====

/**
 * 飞虹 Code (Muse Code 参照复刻)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 经验学习（Experience Learning）— 强化学习式经验系统：
 * - 会话完成后自动提取经验（成功模式、失败教训、高效工具调用序列、自愈修复经验）
 * - 存储到 FH_HOME/experiences/*.jsonl，并以「稳定 id + upsert 合并」实现强化学习
 *   （同一模式被多次验证会累积 sessionCount 与成功率权重，而非无限追加重复记录）
 * - 下次任务开始时用 retrieveRelevantExperiences 做加权检索（标签重叠 + 新鲜度 + 成功率）
 *   注入 system prompt，形成「执行 → 反思 → 回流 → 更强执行」的闭环
 */
import { appendFile, mkdir, readFile, writeFile } from 'fs/promises';
import { existsSync } from 'fs';
import { join } from 'path';
import type { ChatMessage } from '../models/model.interface';
import { logger } from '../shared/logger';
import { classifyError } from './self-heal';

/**
```

```
 * 文件级 Promise 锁：防止两个 run 同时完成时 upsertExperience 互相覆盖。
 * 同一 experiences.jsonl 路径的写入串行化，不同路径互不阻塞。
 */
const fileLocks = new Map<string, Promise<unknown>>();
async function withFileLock<T>(filePath: string, fn: () => Promise<T>): Promise<T> {
  const prev = fileLocks.get(filePath) ?? Promise.resolve();
  let release!: () => void;
  const next = new Promise<void>((resolve) => { release = resolve; });
```

```
28    fileLocks.set(filePath, prev.then(() => next, () => next));
29    try {
30      await prev;
31      return await fn();
32    } finally {
33      release();
34      // 锁队列空时清理 Map 条目，避免内存泄漏
35      if (fileLocks.get(filePath) === next) fileLocks.delete(filePath);
36    }
37  }
38
39  export type ExperienceType =
40    | 'tool-efficiency'      // 高效工具调用模式
41    | 'error-pattern'        // 错误模式与规避
42    | 'path-planning'        // 文件路径规划技巧
43    | 'success-pattern'      // 成功执行序列 / 自愈修复经验
44    | 'performance-tip';     // 性能优化建议
45
46  export interface Experience {
47    id: string;
48    type: ExperienceType;
```

```
  title: string;
  content: string;
  metadata: {
    sessionCount: number;
    successRate: number;
    tags: string[];
    createdAt: string;
    lastUsedAt: string;
  };
}

/** 稳定短哈希（用于经验去重 id） */
function shortHash(s: string): string {
  let h = 5381;
  for (let i = 0; i < s.length; i++) h = ((h << 5) + h + s.charCodeAt(i)) | 0;
  return (h >>> 0).toString(36);
}

/** 由类型 + 归一化键生成稳定 id（跨 run 一致，支撑去重与 usage 追踪） */
export function normalizeExperienceId(type: ExperienceType, key: string): string {
  return `exp-${shortHash(`${type}:${key}`)}`;
```

```
}

/** 从会话历史中提取经验（强化学习素材） */
/** 经验提取器：declare -> (ctx) => Experience | null。表驱动替代长 if 链，便于增删规则 */
interface ExperienceExtractor {
  key: string;
  extract(ctx: ExtractCtx): Experience | null;
}
```

```
78
79 interface ExtractCtx {
80   toolCalls: Array<{ name: string; args: unknown; success: boolean }>;
81   errors: ChatMessage[];
82   mkMeta: (successRate: number, tags: string[]) => Experience['metadata'];
83   now: string;
84 }
85
86 const EXTRACTORS: ExperienceExtractor[] = [
87   {
88     // 经验 1: 高效工具调用模式（成功序列）
89     key: 'tool-efficiency',
90     extract: ({ toolCalls, mkMeta }) => {
91       if (toolCalls.length < 3) return null;
92       const successfulCalls = toolCalls.filter((tc) => tc.success);
93       if (successfulCalls.length < 2) return null;
94       const pattern = successfulCalls.map((tc) => tc.name).join(' → ');
95       return {
96         id: normalizeExperienceId('tool-efficiency', pattern),
97         type: 'tool-efficiency',
98         title: `成功工具序列: ${pattern}`,
```

```
99          content: `在本次任务中，以下工具调用序列成功完成目标:\n${pattern}\n\n建议未来类似任务优先复用此序列，可减少试错轮次。`,
100         metadata: mkMeta(successfulCalls.length / toolCalls.length, successfulCalls.map((tc) => tc.name)),
101       };
102     },
103   },
104   {
105     // 经验 2: 错误模式与规避（用 classifyError 精确归类）
106     key: 'error-pattern',
107     extract: ({ errors, mkMeta }) => {
108       if (errors.length === 0) return null;
109       const categories = new Set<string>();
110       for (const e of errors) {
111         const analysis = classifyError(e.content || '', e.content || '');
112         categories.add(analysis?.category ?? 'unknown');
113       }
114       const uniqueErrors = [...categories];
115       return {
116         id: normalizeExperienceId('error-pattern', uniqueErrors.join(',')),
117         type: 'error-pattern',
118         title: `常见错误模式: ${uniqueErrors.join(', ')}`,
```

```
119         content: `本次任务遇到以下错误类型:\n${uniqueErrors.map((e) => `- ${e}`).join('\n')}\n\n规避建议:提前校验前置条件（路径/权限/依赖），增加超时重试与最小改动原则，避免同类错误重复发生。`,
120         metadata: mkMeta(0, uniqueErrors),
121       };
122     },
123   },
124   {
125     // 经验 3: 干净成功模式（无错误且工具调用高效）
126     key: 'clean-success',
127     extract: ({ toolCalls, errors, mkMeta }) => {
```

// ===== src/models/model-router.ts =====

```
/**
 * 飞虹 Code (Muse Code 参照复刻)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 模型路由器：按策略选优（cost/capability/latency）+ 自动 fallback。
 * 不直接耦合任何供应商，组件注入，便于测试与扩展。
 *
 * M6 增强：
 *  - 模型性能追踪（成功/失败统计）
 *  - 自动择优（基于历史成功率）
 *  - model-stats 命令支持
 *
 * 模型异常自动轮换：
 *  - 上游返回 400/401/403/404（模型/鉴权类）：立即轮换到下一个可用模型，不重试同一模型
 *  - 上游返回 429/500/502/503/504（瞬时/服务端）：轮换下个模型并退避后重试整轮，直到任务完成或达到上限
 *  - 网络/未知错误：按可重试处理，同样触发轮换与重试
 */
import { mkdir, readFile, writeFile } from 'fs/promises';
import { existsSync } from 'fs';
```

```
20 import { join } from 'path';
21 import type { AppConfig } from '../shared/config';
22 import type { CapabilityTag, ModelStrategy } from '../shared/types';
23 import { ModelError } from '../shared/errors';
24 import { logger } from '../shared/logger';
25 import type { ChatRequest, ChatResponse, ModelProvider } from './model.interface';
26 import { OpenAICompatibleProvider } from './providers/openai-compatible.provider';
27 import { OllamaProvider } from './providers/ollama.provider';
28
29 export interface ModelStat {
30   providerId: string;
31   model: string;
32   totalCalls: number;
33   successfulCalls: number;
34   failedCalls: number;
35   totalCostUsd: number;
36   avgLatencyMs: number;
37   lastUsedAt: string;
38   successRate: number;
39 }
40
```

```
41 export interface ModelStatsCache {
42   version: number;
43   stats: ModelStat[];
44   updatedAt: string;
45 }
46
47 const STATS_VERSION = 1;
48 const DEFAULT_STATS_FILE = 'model-stats.jsonl';
```

```
49
50 /** 模型/鉴权类错误：直接轮换到下一个可用模型，重试无意义 */
51 const ROTATE_ONLY_STATUS = new Set<number>([400, 401, 403, 404]);
52 /** 其余状态码（429/500/502/503/504 等瞬时错误）均视为可退避重试 */
53
54 export class ModelRouter {
55   private readonly providers: ModelProvider[];
56   private readonly strategy: ModelStrategy;
57   private readonly budget: number;
58   private readonly statsFile: string;
59   private readonly statsHomeDir: string;
60   private readonly maxRetries: number;
61   private readonly backoffMs: number;
62   private statsCache: ModelStatsCache | null = null;
63
64   constructor(
65     providers: ModelProvider[],
66     strategy: ModelStrategy,
67     budget: number,
68     statsFile: string = '',
69     statsHomeDir: string = '',
```

```
70      maxRetries: number = 3,
71      backoffMs: number = 1000,
72    ) {
73      this.providers = providers;
74      this.strategy = strategy;
75      this.budget = budget;
76      this.statsFile = statsFile || DEFAULT_STATS_FILE;
77      this.statsHomeDir = statsHomeDir;
78      this.maxRetries = Math.max(1, maxRetries);
79      this.backoffMs = Math.max(0, backoffMs);
80    }
81
82    /** 从 AppConfig 构建（默认路由，联调真实模型时使用）；homeDir 用于统计自动落盘 */
83    static fromConfig(cfg: AppConfig, statsFile?: string): ModelRouter {
84      const providers = cfg.models.providers.map((p) =>
85        p.type === 'ollama' ? new OllamaProvider(p) : new OpenAICompatibleProvider(p),
86      );
87      return new ModelRouter(
88        providers,
89        cfg.models.defaultStrategy,
90        cfg.models.budgetPerTaskUsd,
```

```
91        statsFile,
92        cfg.app.homeDir,
93        cfg.runtime.maxRetries,
94      );
95    }
96
97    /** 加载性能统计缓存 */
98    async loadStats(homeDir: string): Promise<void> {
```

```
99    const file = join(homeDir, this.statsFile);
100   if (!existsSync(file)) {
101     this.statsCache = { version: STATS_VERSION, stats: [], updatedAt: new Date().toISOString() };
102     return;
103   }
104   try {
105     const content = await readFile(file, 'utf8');
106     const lines = content.trim().split('\n').filter(Boolean);
107     const stats: ModelStat[] = [];
108     for (const line of lines) {
109       try {
110         stats.push(JSON.parse(line) as ModelStat);
111       } catch {
112         // 跳过损坏记录
113       }
114     }
115     this.statsCache = { version: STATS_VERSION, stats, updatedAt: new Date().toISOString() };
116   } catch {
117     this.statsCache = { version: STATS_VERSION, stats: [], updatedAt: new Date().toISOString() };
118   }
119 }
```

```
120
121   /** 保存性能统计 */
122   async saveStats(homeDir: string): Promise<void> {
123     if (!this.statsCache) return;
124     const file = join(homeDir, this.statsFile);
125     await mkdir(homeDir, { recursive: true });
126     const lines = this.statsCache.stats.map((s) => JSON.stringify(s)).join('\n') + '\n';
127     await writeFile(file, lines, 'utf8');
128   }
129
130   /** 更新模型调用统计（并在配置了 homeDir 时同步落盘，供 model-stats 命令读取） */
131   async updateStat(providerId: string, model: string, success: boolean, costUsd: number, latencyMs: nu
    mber): Promise<void> {
132     if (!this.statsCache) {
133       this.statsCache = { version: STATS_VERSION, stats: [], updatedAt: new Date().toISOString() };
134     }
135
136     const existing = this.statsCache.stats.find((s) => s.providerId === providerId && s.model === mode
    l);
137     if (existing) {
138       existing.totalCalls++;
139       if (success) existing.successfulCalls++;
```

```
140       else existing.failedCalls++;
141       existing.totalCostUsd += costUsd;
142       existing.avgLatencyMs = (existing.avgLatencyMs * (existing.totalCalls - 1) + latencyMs) / existi
    ng.totalCalls;
143       existing.lastUsedAt = new Date().toISOString();
144       existing.successRate = existing.successfulCalls / existing.totalCalls;
145     } else {
146       this.statsCache.stats.push({
147         providerId,
148         model,
```

```
149         totalCalls: 1,
150         successfulCalls: success ? 1 : 0,
151         failedCalls: success ? 0 : 1,
152         totalCostUsd: costUsd,
153         avgLatencyMs: latencyMs,
154         lastUsedAt: new Date().toISOString(),
155         successRate: success ? 1 : 0,
156       });
157     }
158     this.statsCache.updatedAt = new Date().toISOString();
159
160     // M6 闭环修复：统计仅驻留内存则 model-stats 永远读不到数据，这里同步落盘
161     if (this.statsHomeDir) {
162       try {
163         await this.saveStats(this.statsHomeDir);
164       } catch (e) {
165         logger.warn('failed to persist model stats', {
166           error: e instanceof Error ? e.message : String(e),
167         });
168       }
169     }
```

```
170   }
171
172   /** 获取模型统计报告 */
173   getStats(): ModelStat[] {
174     return this.statsCache?.stats ?? [];
175   }
176
177   /** 按策略排序后的 provider 列表（含能力过滤与 fallback 顺序） */
178   rank(tags?: CapabilityTag[]): ModelProvider[] {
179     let list = [...this.providers];
180     if (tags && tags.length > 0) {
181       const filtered = list.filter((p) => tags.every((t) => p.tags.includes(t)));
182       if (filtered.length > 0) list = filtered;
183     }
184     return list.sort((a, b) => this.score(b) - this.score(a));
185   }
186
187   private score(p: ModelProvider): number {
188     let base = 0;
189     if (this.strategy === 'cost') base = -(p.costPer1k ?? 0);
190     else if (this.strategy === 'latency') base = p.tags.includes('local') ? 1 : 0;
```

```
191     else base = (p.tags.includes('reasoning') ? 2 : 0) + (p.tags.includes('code-gen') ? 1 : 0);
192
193     // M6: 加权历史成功率（最多 0.3 分）；统计按 providerId+model 键控，查找需一致
194     if (this.statsCache) {
195       const stat = this.statsCache.stats.find((s) => s.providerId === p.id && s.model === p.model);
196       if (stat && stat.totalCalls >= 3) {
197         base += stat.successRate * 0.3;
198       }
```

```
    }
    return base;
  }

  /**
   * 从错误中提取上游 HTTP 状态码（优先 ModelError.statusCode，兼容从消息解析 "HTTP <n>"）。
   * 网络/未知错误返回 undefined。
   */
  private statusOf(e: unknown): number | undefined {
    if (e instanceof ModelError) return e.statusCode;
    if (e instanceof Error) {
      const m = /HTTP\s+(\d{3})/.exec(e.message);
      if (m) return Number(m[1]);
    }
    return undefined;
  }

  /** 该错误是否值得退避重试（模型/鉴权类 400/401/403/404 不重试，其余默认重试） */
  private isRetryable(e: unknown): boolean {
    const status = this.statusOf(e);
    if (status === undefined) return true; // 网络/未知错误默认可重试
```

```
220     return !ROTATE_ONLY_STATUS.has(status);
221   }
222
223   private sleep(ms: number): Promise<void> {
224     return ms > 0 ? new Promise((r) => setTimeout(r, ms)) : Promise.resolve();
225   }
226
227   /**
228    * 调用并自动轮换。按状态码分类：
229    *   - 400/401/403/404（模型/鉴权类）：立即轮换下个可用模型，不做退避重试
230    *   - 429/500/502/503/504（瞬时/服务端）：轮换下个模型，整轮结束后退避重试，直到任务完成或达到上限
231    *   - 网络/未知错误：按可重试处理
232    * 全部 provider 均不可用时抛出最后错误。
233    */
234   async chat(req: ChatRequest, tags?: CapabilityTag[]): Promise<ChatResponse> {
235     const order = this.rank(tags);
236     if (order.length === 0) {
237       throw new ModelError('未配置任何可用模型 provider', 'router', 500);
238     }
239
240     const maxCycles = this.maxRetries;
```

```
    let lastErr: unknown;
    let lastRetryable = true;

    for (let cycle = 0; cycle < maxCycles; cycle++) {
      for (const p of order) {
        const startTime = Date.now();
        try {
          const resp = await p.chat(req);
```

```
249           const latency = Date.now() - startTime;
250
251           // M6: 更新性能统计（成功记录响应模型名）
252           await this.updateStat(p.id, resp.model, true, resp.costUsd, latency);
253
254           if (this.budget > 0 && resp.costUsd > this.budget) {
255             logger.warn('cost over budget', { provider: p.id, cost: resp.costUsd, budget: this.budget
    });
256           }
257           return resp;
258         } catch (e) {
259           const latency = Date.now() - startTime;
260           // M6: 记录失败（用 provider 配置的模型名，避免统计出 model='' 的空条目）
261           await this.updateStat(p.id, p.model, false, 0, latency);
262           const status = this.statusOf(e);
263           lastErr = e;
264           lastRetryable = this.isRetryable(e);
265           logger.warn('模型调用失败，自动轮换下个模型', {
266             provider: p.id,
267             status,
268             cycle: cycle + 1,
```

```
            error: e instanceof Error ? e.message : String(e),
          });
        }
      }

      // 整轮结束：若原因是模型/鉴权类（400/401 等），退避重试无意义，直接终止
      if (!lastRetryable) break;
      // 瞬时错误（429/500 等）：退避后重试下一轮，直到任务完成或达到上限
      if (cycle < maxCycles - 1) {
        await this.sleep(this.backoffMs * Math.pow(2, cycle));
      }
    }

    throw lastErr instanceof Error ? lastErr : new ModelError('所有可用模型均调用失败', 'router', 500);
  }
}


// ===== src/models/model.interface.ts =====

/**
```

```
 * 飞虹 Code (Muse Code 参照复刻)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * 模型层接口契约：与具体 LLM 供应商解耦
 */
import type { CapabilityTag } from '../shared/types';

export type Role = 'system' | 'user' | 'assistant' | 'tool';
```

```
10
11 /** assistant 发起的一次工具调用 */
12 export interface ToolCall {
13   id: string;
14   name: string;
15   /** 归一后的 JSON 参数对象（供应商原始可能是字符串，provider 负责 parse） */
16   arguments: Record<string, unknown>;
17 }
18
19 export interface ChatMessage {
20   role: Role;
21   content: string;
22   /** 工具结果回填时对应 assistant 的 tool_call id */
23   toolCallId?: string;
24   /** assistant 回合发起的工具调用 */
25   toolCalls?: ToolCall[];
26 }
27
28 /** 暴露给模型的工具定义（JSON Schema 描述入参） */
29 export interface ToolDefinition {
30   name: string;
```

```
31   description: string;
32   parameters: Record<string, unknown>;
33 }
34
35 export interface ChatRequest {
36   messages: ChatMessage[];
37   tools?: ToolDefinition[];
38   temperature?: number;
39   maxTokens?: number;
40   timeoutMs?: number;
41   /** 外部中断信号（如任务队列的停止按钮），触发后中断当前网络请求与编排循环 */
42   signal?: AbortSignal;
43 }
44
45 export interface TokenUsage {
46   promptTokens: number;
47   completionTokens: number;
48   totalTokens: number;
49 }
50
51 export interface ChatResponse {
```

```
52   message: ChatMessage;
53   usage: TokenUsage;
54   providerId: string;
55   model: string;
56   costUsd: number;
57 }
58
59 export interface ModelProvider {
```

```
  readonly id: string;
  readonly model: string;
  readonly tags: CapabilityTag[];
  readonly costPer1k?: number;
  chat(req: ChatRequest): Promise<ChatResponse>;
}

```

// ===== src/models/providers/openai-compatible.provider.ts =====

```
/**
 * 飞虹 Code (Muse Code 参照复刻)
 * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
 *
 * OpenAI 兼容供应商（DeepSeek / 通义 / OpenRouter 等 OpenAI 协议接口）
 */
import { ModelError } from '../../shared/errors';
import type { ProviderConfig } from '../../shared/config';
import type { CapabilityTag } from '../../shared/types';
import type { ChatRequest, ChatResponse, ModelProvider, ToolDefinition } from '../model.interface';
import { openAIChatResponseSchema, parseToolArgs } from '../model.dto';
```

```ts
import { estimateCost } from '../cost';

export class OpenAICompatibleProvider implements ModelProvider {
  readonly id: string;
  readonly model: string;
  readonly tags: CapabilityTag[];
  readonly costPer1k: number;
  private readonly baseURL: string;
  private readonly apiKey?: string;

  constructor(cfg: ProviderConfig) {
    this.id = cfg.id;
    this.model = cfg.model ?? cfg.id;
    this.tags = cfg.tags;
    this.costPer1k = cfg.costPer1k ?? 0;
    this.baseURL = cfg.baseURL.replace(/\/$/, '');
    this.apiKey = cfg.apiKey;
  }

  async chat(req: ChatRequest): Promise<ChatResponse> {
    const body = {
```

```
      model: this.model,
      messages: req.messages.map((m) => ({
        role: m.role,
        content: m.content,
        tool_calls: m.toolCalls?.map((t) => ({
          id: t.id,
          type: 'function',
          function: { name: t.name, arguments: JSON.stringify(t.arguments) },
```

```
41        })),
42        tool_call_id: m.toolCallId,
43      })),
44      tools:
45        req.tools && req.tools.length > 0
46          ? req.tools.map((t: ToolDefinition) => ({
47              type: 'function',
48              function: { name: t.name, description: t.description, parameters: t.parameters },
49            }))
50          : undefined,
51      temperature: req.temperature ?? 0,
52      max_tokens: req.maxTokens ?? 4096,
53    };
54
55    const controller = new AbortController();
56    let timer: NodeJS.Timeout | undefined;
57    if (req.timeoutMs && req.timeoutMs > 0) {
58      timer = setTimeout(() => controller.abort(), req.timeoutMs);
59    }
60    // 外部中断信号（任务停止）：链接到本地 controller，触发即中断 fetch
61    const onExternalAbort = (): void => controller.abort();
```

```
62    if (req.signal) {
63      if (req.signal.aborted) controller.abort();
64      else req.signal.addEventListener('abort', onExternalAbort, { once: true });
65    }
66
67    let res: Response;
68    try {
69      res = await fetch(`${this.baseURL}/chat/completions`, {
70        method: 'POST',
71        headers: {
72          'Content-Type': 'application/json',
73          ...(this.apiKey ? { Authorization: `Bearer ${this.apiKey}` } : {}),
74        },
75        body: JSON.stringify(body),
76        signal: controller.signal,
77      });
78    } catch (e) {
79      const aborted = controller.signal.aborted;
80      if (aborted) throw new ModelError('任务已被用户中断', this.id);
81      throw new ModelError(`网络请求失败: ${e instanceof Error ? e.message : String(e)}`, this.id);
82    } finally {
```

```
83        if (timer) clearTimeout(timer);
84        if (req.signal) req.signal.removeEventListener('abort', onExternalAbort);
85      }
86
87      if (!res.ok) {
88        const text = await res.text().catch(() => '');
89        throw new ModelError(`HTTP ${res.status} ${text.slice(0, 200)}`, this.id, res.status);
90      }
```

```
91
92      const json: unknown = await res.json().catch(() => undefined);
93      const parsed = openAIChatResponseSchema.safeParse(json);
94      if (!parsed.success) {
95        throw new ModelError(`响应结构校验失败: ${parsed.error.message}`, this.id);
96      }
97
98      const choice = parsed.data.choices[0];
99      const usage = parsed.data.usage
100       ? {
101           promptTokens: parsed.data.usage.prompt_tokens,
102           completionTokens: parsed.data.usage.completion_tokens,
103           totalTokens: parsed.data.usage.total_tokens,
104         }
105       : { promptTokens: 0, completionTokens: 0, totalTokens: 0 };
106
107     const toolCalls = choice.message.tool_calls?.map((tc) => ({
108       id: tc.id,
109       name: tc.function.name,
110       arguments: parseToolArgs(tc.function.arguments),
111     }));
```

```
112
113     return {
114       message: { role: 'assistant', content: choice.message.content ?? '', toolCalls },
115       usage,
116       providerId: this.id,
117       model: this.model,
118       costUsd: estimateCost(usage, this.costPer1k),
119     };
120   }
121 }
122
```

// ===== src/web/server.ts =====

```
1 /**
2  * 飞虹 Code (Muse Code 参照复刻)
3  * 晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人：吴赐虹
4  *
5  * M5 Web 控制台：自包含 Express 服务骨架 + 多视图 API。
6  *   - 任务队列（P4-1 / P5-2 / P6-4）
7  *   - 技能市场（插件市场）：聚合 ClawHub + Agent-Foundry
```

```
 *    - 自动化：快捷指令集（一键发起任务）
 *    - 模板库：内置 + 用户自定义
 *    - 办公助理：文档处理能力清单
 *    - 登录：手机号直登（无短信验证，本地会话令牌）
 *    - 文件/工作区：右侧任务详情可直接打开文件夹、浏览器、预览文件
 * 鉴权：Bearer Token（fail-closed），主令牌（FH_WEB_TOKEN）与会话令牌并行。
 */
import express, { type Request, type Response } from 'express';
```

```
import { randomBytes, randomUUID } from 'crypto';
import { join, resolve, relative, isAbsolute, dirname } from 'path';
import {
  existsSync,
  readFileSync,
  writeFileSync,
  mkdirSync,
  readdirSync,
  statSync,
  lstatSync,
  renameSync,
  unlinkSync,
} from 'fs';
import { spawn } from 'child_process';
import { requireToken, SessionStore, type Session, WELCOME_TASKS } from './auth';
import {
  TaskQueue,
  type TaskRecord,
  type TaskStep,
  type AgentType,
  type TaskPermissions,
```

```
} from './task-queue';
import { VERSION, PRODUCT, SIGNATURE } from '../cli/version';
import { t, getLang } from '../shared/i18n';
import { isEnterpriseEnabled } from '../enterprise';
import { resolveHomeDir } from '../shared/config';
import {
  getMemoryConfig,
  readShortTerm,
  readLongTerm,
  getMemoryStats,
} from '../memory';
import {
  getSummaryHistory,
  summarizeMemory,
} from '../memory/auto-summarize';
import {
  encryptText,
  decryptText,
  isEncrypted,
  getMasterKey,
  getRsaKeys,
```

```
  rsaDecrypt,
} from '../shared/secure-store';

export interface ServeOptions {
  port?: number;
  token?: string;
}

```

```
/**
 * 启动 Web 控制台。
 * - 端口：opts.port > FH_WEB_PORT > 8080
 * - 令牌：opts.token > FH_WEB_TOKEN > 自动生成（仅本次会话有效，fail-closed）
 */
export function startWebServer(opts: ServeOptions = {}): {
  port: number;
  token: string;
  url: string;
  close: () => void;
} {
  // Web 环境无法进行交互式审批，禁用审批检查
  if (process.env.FH_REQUIRE_APPROVAL === undefined) {
    process.env.FH_REQUIRE_APPROVAL = 'false';
  }

  const port = opts.port ?? Number(process.env.FH_WEB_PORT ?? 8080);
  let token = opts.token ?? process.env.FH_WEB_TOKEN ?? '';
  if (!token) {
    token = randomBytes(24).toString('hex');
    console.log(t('serve.tokenAuto', { token }));
```

```
}

const app = express();
// 放宽请求体上限以支持截图/图片 base64 上传（8MB），仍可有效防御内存耗尽
app.use(express.json({ limit: '8mb' }));
// 静态仪表盘（开发时直接从 src 读取，避免 dist 被锁）
const publicDir =
  process.env.FH_WEB_SRC_PUBLIC || join(__dirname, 'public');
app.use(express.static(publicDir, { maxAge: '0' }));

// 登录会话存储（进程内）
const sessions = new SessionStore();
// 当前 Web 控制台工作区，任务提交缺省时使用
let serverWorkspaceDir = resolve(process.cwd());

// 任务队列（进程内；服务端静默执行）— 需在登录接口前初始化
const persistDir =
  process.env.FH_TASK_PERSIST_DIR?.trim() ||
  join(process.env.FH_HOME?.trim() || join(require('os').homedir(), '.feihong-code'), 'tasks');
const queue = new TaskQueue({
  concurrency: Number(process.env.FH_TASK_CONCURRENCY ?? 2),
```

```
108     webhookUrl: process.env.FH_TASK_WEBHOOK_URL,
109     persistDir,
110   });
111
112   // 公开健康检查（仅暴露版本/状态等观测信息，无敏感数据）
113   app.get('/api/health', (_req: Request, res: Response) => {
114     res.json({
115       ok: true,
```

```
116       product: PRODUCT,
117       version: VERSION,
118       signature: SIGNATURE,
119       enterprise: isEnterpriseEnabled(),
120       lang: getLang(),
121       time: new Date().toISOString(),
122     });
123   });
124
125   // 手机号直登：无短信验证，生成本地会话令牌
126   app.post('/api/auth/login', (req: Request, res: Response) => {
127     const body = (req.body ?? {}) as Record<string, any>;
128     const phone = typeof body?.phone === 'string' ? body.phone.trim() : '';
129     if (!phone) {
130       res.status(400).json({ ok: false, error: '请输入手机号码' });
131       return;
132     }
133     // 使用新的登录方法，支持首次登录检测
134     const result = sessions.login(phone);
135     const session = sessions.get(result.token);
136
```

```
137     // 如果是首次登录，自动创建引导任务
138     let welcomeTasks: any[] = [];
139     if (result.isFirstLogin) {
140       // 标记会话为首次登录（用于后续接口识别）
141       if (session) {
142         session.isFirstLogin = true;
143       }
144
145       // 创建引导任务
146       for (const task of WELCOME_TASKS) {
147         const record = queue.submit(task.goal, {
148           workspaceDir: serverWorkspaceDir,
149           agentType: 'general' as const,
150         });
151         welcomeTasks.push({
152           ...task,
153           taskId: record.id,
154           status: record.status,
155         });
156       }
157     }
```

```
158
159     res.json({
160       ok: true,
161       token: result.token,
162       phone,
163       isFirstLogin: result.isFirstLogin,
164       welcomeTasks,
165     });
```

```
166  });
167  // 鉴权：其余 /api 需 Bearer token（静态资源与 login/health 除外）
168  // 公开 API（无需认证）
169  app.get('/api/drives', (_req: Request, res: Response) => {
170    res.json({ ok: true, drives: getAvailableDrives() });
171  });
172
173  // 记忆管理 API（公开访问，无敏感信息）
174  const memoryConfig = getMemoryConfig();
175  app.get('/api/memory/short', (_req: Request, res: Response) => {
176    const date = typeof _req.query.date === 'string' ? _req.query.date : undefined;
177    const content = readShortTerm(memoryConfig, date ? new Date(date) : undefined);
178    res.json({ ok: true, content, date: date || new Date().toISOString().split('T')[0] });
179  });
180  app.get('/api/memory/long', (_req: Request, res: Response) => {
181    const content = readLongTerm(memoryConfig);
182    res.json({ ok: true, content });
183  });
184  app.get('/api/memory/stats', (_req: Request, res: Response) => {
185    const stats = getMemoryStats(memoryConfig);
186    res.json({ ok: true, ...stats });
```

```
});
app.post('/api/memory/summarize', async (req: Request, res: Response) => {
  const body = req.body as Record<string, any>;
  const forceDate = typeof body?.date === 'string' ? body.date : undefined;
  try {
    const result = await summarizeMemory(memoryConfig, forceDate);
    res.json(result);
  } catch (e: any) {
    res.status(500).json({ ok: false, error: e.message });
  }
});
app.get('/api/memory/history', (_req: Request, res: Response) => {
  const limit = typeof _req.query.limit === 'string' ? parseInt(_req.query.limit) || 30 : 30;
  const history = getSummaryHistory(memoryConfig, limit);
  res.json({ ok: true, history });
});

app.use('/api', requireToken(token, sessions));

app.get('/api/auth/me', (req: Request, res: Response) => {
  const session = (req as Request & { user?: Session }).user;
```

```
208     res.json({
209       ok: true,
210       phone: session?.phone ?? null,
211       isFirstLogin: session?.isFirstLogin ?? false,
212     });
213   });
214
215   app.post('/api/tasks', (req: Request, res: Response) => {
```

```
216     const body = req.body as Record<string, any>;
217     const goal = typeof body?.goal === 'string' ? body.goal.trim() : '';
218     if (!goal) {
219       res.status(400).json({ ok: false, error: '缺少 goal 字段' });
220       return;
221     }
222     const agentType: AgentType | undefined =
223       typeof body?.agentType === 'string' ? (body.agentType as AgentType) : undefined;
224     const permissions: TaskPermissions | undefined =
225       typeof body?.permissions === 'object' && body?.permissions !== null
226         ? (body.permissions as TaskPermissions)
227         : undefined;
228     const workspaceDir =
229       typeof body?.workspaceDir === 'string' && body.workspaceDir.trim()
230         ? body.workspaceDir.trim()
231         : serverWorkspaceDir;
232     const modelId =
233       typeof body?.modelId === 'string' && body.modelId.trim() ? body.modelId.trim() : undefined;
234     // 附件：前端暂存区统一上传后的文件路径列表
235     const attachments: string[] = Array.isArray(body?.attachments)
```

```
236       ? (body.attachments as unknown[]).filter((x) => typeof x === 'string' && x.trim()).map((x) => (x as string).trim())
237       : [];
238     const record = queue.submit(goal, { modelId, workspaceDir, agentType, permissions, attachments });
239     res.status(201).json({ ok: true, task: publicTask(record, true) });
240   });
241   app.get('/api/tasks', (_req: Request, res: Response) => {
242     res.json({ ok: true, tasks: queue.list().map((t) => publicTask(t, false)) });
243   });
244   app.get('/api/tasks/:id', (req: Request, res: Response) => {
245     const record = queue.get(req.params.id);
246     if (!record) {
247       res.status(404).json({ ok: false, error: '任务不存在' });
248       return;
249     }
250     res.json({ ok: true, task: publicTask(record, true) });
251   });
252   // 多轮续接：向当前任务追加一条用户消息，所有对话归属同一任务生命周期
253   app.post('/api/tasks/:id/messages', (req: Request, res: Response) => {
254     const body = req.body as Record<string, any>;
255     const message = typeof body?.message === 'string' ? body.message.trim() : '';
256     if (!message) {
```

```
257       res.status(400).json({ ok: false, error: '缺少 message 字段' });
258       return;
259     }
260     // 附件：前端暂存区统一上传后的文件路径列表
261     const attachments: string[] = Array.isArray(body?.attachments)
262       ? (body.attachments as unknown[]).filter((x) => typeof x === 'string' && x.trim()).map((x) => (x as string).trim())
263       : [];
264     const record = queue.continueTask(req.params.id, message, attachments);
265     if (!record) {
```

```
266       res.status(409).json({ ok: false, error: '任务不存在或正在执行中，请等待完成后再继续对话' });
267       return;
268     }
269     res.status(201).json({ ok: true, task: publicTask(record, true) });
270   });
271   app.delete('/api/tasks/:id', (req: Request, res: Response) => {
272     const success = queue.delete(req.params.id);
273     if (!success) {
274       res.status(400).json({ ok: false, error: '无法删除运行中的任务' });
275       return;
276     }
277     res.json({ ok: true });
278   });
279   // P9：停止单个指定任务（精准中止，不影响其他运行中的任务）
280   app.post('/api/tasks/:id/stop', (req: Request, res: Response) => {
281     const ok = queue.cancelTask(req.params.id);
282     if (!ok) {
283       res.status(409).json({ ok: false, error: '任务不存在或已结束，无法停止' });
284       return;
285     }
286     res.json({ ok: true, taskId: req.params.id, status: 'failed' });
```

```
});
// P9：停止所有任务（中断运行 + 清空队列，后续可继续提交新任务）
app.post('/api/tasks/stop', (_req: Request, res: Response) => {
  queue.cancel();
  res.json({ ok: true });
});

// P5-2：webhook 注册/查询
app.post('/api/webhook', (req: Request, res: Response) => {
  const body = req.body as Record<string, any>;
  const url = typeof body?.url === 'string' ? body.url.trim() : '';
  if (!url) {
    res.status(400).json({ ok: false, error: '缺少 url 字段' });
    return;
  }
  queue.setWebhookUrl(url);
  res.json({ ok: true, webhookUrl: url });
});
app.get('/api/webhook', (_req: Request, res: Response) => {
  res.json({ ok: true, webhookUrl: queue.getWebhookUrl() || null });
});
```

```
308
309 /* ========== 持久化辅助 ========== */
310 const homeDir = resolveHomeDir();
311 // 三重加密密钥体系：主密钥（AES 存储加密）+ RSA 密钥对（通信加密）
312 const masterKey = getMasterKey(homeDir);
313 const rsaKeys = getRsaKeys(homeDir);
314 function loadJsonFile<T>(file: string, fallback: T): T {
315   try {
```

```
316       if (!existsSync(file)) return fallback;
317       return JSON.parse(readFileSync(file, 'utf8')) as T;
318     } catch {
319       return fallback;
320     }
321   }
322   /** 同步睡眠（避免引入异步复杂度；用 Atomics.wait 兼容 Node 12+） */
323   function sleepSync(ms: number): void {
324     Atomics.wait(new Int32Array(new SharedArrayBuffer(4)), 0, 0, ms);
325   }
326
327   /**
328    * 原子写入 JSON 文件（tmp → renameSync）。
329    * - 真原子替换：rename 成功即生效，tmp 不会残留（区别于旧的"写 tmp 再写 file 再删 tmp"，
330    *   旧方案在 Windows 下高频写会偶发 EPERM：Defender/句柄锁住 .tmp 文件导致 open 失败，
331    *   且崩溃时 tmp 残留，下次 open 又被锁 → 连锁失败）。
332    * - EPERM 瞬时锁：重试 3 次（50ms/100ms/150ms 退避）。
333    * - 绝不向上抛：失败返回 false，由调用方决定降级策略（内存态 / 500 响应）。
334    */
335   function saveJsonFile(file: string, data: unknown): boolean {
336     try {
```

```
337       mkdirSync(dirname(file), { recursive: true });
338     } catch {
339       /* ignore */
340     }
341     // tmp 文件名带 PID：进程唯一，避免与历史残留（如被外部句柄锁住的 .tmp）或并发进程冲突
342     const tmp = file + '.' + process.pid + '.tmp';
343     for (let attempt = 0; attempt < 3; attempt++) {
344       try {
345         writeFileSync(tmp, JSON.stringify(data, null, 2), 'utf8');
346         renameSync(tmp, file);
347         return true;
348       } catch (e) {
349         // 清理残留 tmp（unlink 也可能 EPERM，忽略）
350         try {
351           unlinkSync(tmp);
352         } catch {
353           /* ignore */
354         }
355         if (attempt === 2) {
356           console.warn('[fhcode] 配置文件写入失败（已重试 3 次，保持内存态）', file, (e as Error)?.messag
    e);
357           return false;
```

```
358         }
359         sleepSync(50 * (attempt + 1));
360       }
361     }
362     return false;
363   }
364
365   /* ========== 路径安全 ========== */
```

```
366 // 获取系统所有可用的 Windows 驱动器列表（同步，使用 fs.existsSync 检测）
367 function getAvailableDrives(): string[] {
368   if (process.platform !== 'win32') return ['/'];
369   const drives: string[] = [];
370   for (let code = 65; code <= 90; code++) {
371     const drive = String.fromCharCode(code) + ':\\';
372     if (existsSync(drive)) drives.push(drive);
373   }
374   return drives.length > 0 ? drives : [];
375 }
376
377 // 驱动器根目录只在启动时探测一次并缓存。
378 // 历史缺陷：此处曾对每次路径校验 spawn 一个 `cmd /c wmic logicaldisk`，
379 // 而 wmic 的 close 回调在函数 return 之后才触发（结果根本用不上），
380 // 等于每次浏览目录都白起一个进程；Win11 已移除 wmic，spawn 失败还会拖慢/打断请求。
381 // 现改为同步 existsSync 探测 + 进程级缓存，零子进程。
382 const driveRootsCache: string[] = getAvailableDrives();
383
384 const allowedRoots = (): string[] => [
385   resolve(serverWorkspaceDir),
386   resolve(homeDir),
```

```
387     resolve(process.cwd()),
388     ...driveRootsCache,
389   ];
390
391   function isPathAllowed(target: string): boolean {
392     const resolved = resolve(target);
393     for (const root of allowedRoots()) {
394       const rel = relative(root, resolved);
395       if (rel === '' || (!rel.startsWith('..') && !isAbsolute(rel))) {
396         return existsSync(resolved);
397       }
398     }
399     return false;
400   }
401
402   function assertPathAllowed(target: string, res: Response): boolean {
403     if (!isPathAllowed(target)) {
404       res.status(403).json({ ok: false, error: '路径不在允许范围内或不存在' });
405       return false;
406     }
407     return true;
```

```
}

/* ========== 工作区与文件浏览 ========== */
app.get('/api/workspace', (_req: Request, res: Response) => {
  res.json({ ok: true, cwd: serverWorkspaceDir });
});
app.post('/api/workspace', (req: Request, res: Response) => {
  const body = (req.body ?? {}) as Record<string, any>;
```

```
416     const cwd = typeof body?.cwd === 'string' ? body.cwd.trim() : '';
417     if (!cwd) {
418       res.status(400).json({ ok: false, error: '缺少 cwd 字段' });
419       return;
420     }
421     const resolved = resolve(cwd);
422     if (!existsSync(resolved) || !statSync(resolved).isDirectory()) {
423       res.status(400).json({ ok: false, error: '目录不存在' });
424       return;
425     }
426     serverWorkspaceDir = resolved;
427     res.json({ ok: true, cwd: serverWorkspaceDir });
428   });
429
430   app.get('/api/workspace/list', (req: Request, res: Response) => {
431     const raw = typeof req.query.path === 'string' ? req.query.path.trim() : '';
432     // path 为空 / '.' 时回落到服务端工作区，避免 resolve('.') 指向进程 cwd 造成困惑
433     const rawPath = !raw || raw === '.' ? serverWorkspaceDir : raw;
434     const dir = resolve(rawPath);
435     if (!assertPathAllowed(dir, res)) return;
436     try {
```

```
437       // withFileTypes 失败时（部分网络盘/权限目录）退回普通 readdir
438       const names = readdirSync(dir);
439       const entries = names
440         .map((name) => {
441           const full = join(dir, name);
442           try {
443             const st = lstatSync(full);
444             return {
445               name,
446               path: full,
447               type: st.isDirectory() ? 'dir' : st.isFile() ? 'file' : 'other',
448             };
449           } catch {
450             return null;
451           }
452         })
453         .filter(Boolean);
454       res.json({ ok: true, cwd: dir, entries });
455     } catch (e) {
456       res.status(500).json({ ok: false, error: '读取目录失败: ' + (e as Error).message });
457     }
```

```
});

app.post('/api/files/read', (req: Request, res: Response) => {
  const body = (req.body ?? {}) as Record<string, any>;
  const file = typeof body?.path === 'string' ? body.path.trim() : '';
  if (!file || !assertPathAllowed(file, res)) return;
  try {
    const st = statSync(file);
```

```
466       if (!st.isFile()) {
467         res.status(400).json({ ok: false, error: '不是文件' });
468         return;
469       }
470       if (st.size > 2 * 1024 * 1024) {
471         res.status(400).json({ ok: false, error: '文件超过 2MB，建议用本地编辑器打开' });
472         return;
473       }
474       const content = readFileSync(file, 'utf8');
475       res.json({ ok: true, path: file, content });
476     } catch (e) {
477       res.status(500).json({ ok: false, error: '读取失败: ' + (e as Error).message });
478     }
479   });
480
481   /* ========== 打开本地文件夹/浏览器 ========== */
482   app.post('/api/open/folder', (req: Request, res: Response) => {
483     const body = (req.body ?? {}) as Record<string, any>;
484     const dir = typeof body?.path === 'string' ? body.path.trim() : serverWorkspaceDir;
485     if (!dir || !assertPathAllowed(dir, res)) return;
486     try {
```

```
487       openFolder(dir);
488       res.json({ ok: true });
489     } catch (e) {
490       res.status(500).json({ ok: false, error: '打开失败: ' + (e as Error).message });
491     }
492   });
493
494   app.post('/api/open/browser', (req: Request, res: Response) => {
495     const body = (req.body ?? {}) as Record<string, any>;
496     const url = typeof body?.url === 'string' ? body.url.trim() : '';
497     if (!url) {
498       res.status(400).json({ ok: false, error: '缺少 url 字段' });
499       return;
500     }
501     try {
502       openBrowser(url);
503       res.json({ ok: true });
504     } catch (e) {
505       res.status(500).json({ ok: false, error: '打开失败: ' + (e as Error).message });
506     }
507   });
```

```
508
509 /* ========== 上传文件/图片 ========== */
510 const uploadsDir = () => join(homeDir, 'uploads');
511 app.post('/api/upload', (req: Request, res: Response) => {
512   const body = (req.body ?? {}) as Record<string, any>;
513   const name = typeof body?.name === 'string' ? body.name.trim() : '';
514   const mime = typeof body?.mime === 'string' ? body.mime.trim() : 'application/octet-stream';
515   const data = typeof body?.dataBase64 === 'string' ? body.dataBase64.trim() : '';
```

```
516     if (!name || !data) {
517       res.status(400).json({ ok: false, error: '缺少 name 或 dataBase64 字段' });
518       return;
519     }
520     try {
521       mkdirSync(uploadsDir(), { recursive: true });
522       const safeName = name.replace(/[^a-zA-Z0-9_.\-]/g, '_');
523       const dest = join(uploadsDir(), `${Date.now()}_${safeName}`);
524       writeFileSync(dest, Buffer.from(data, 'base64'));
525       res.json({ ok: true, path: dest, name, mime });
526     } catch (e) {
527       res.status(500).json({ ok: false, error: '上传失败: ' + (e as Error).message });
528     }
529   });
530
531   /* ========== 技能市场（插件市场）：聚合 ClawHub + Agent-Foundry ========== */
532   const CLAWHUB_API = 'https://clawhub.ai/api/v1/skills';
533   const AGENT_FOUNDRY_CATALOG =
534     'https://raw.githubusercontent.com/hebertzhu/agent-foundry/main/catalog/skills-catalog.json';
535
536   async function fetchWithRetry(url: string, timeoutMs = 15000, retries = 1): Promise<any> {
```

```
537     let lastErr: unknown;
538     for (let attempt = 0; attempt <= retries; attempt++) {
539       const ctrl = new AbortController();
540       const timer = setTimeout(() => ctrl.abort(), timeoutMs);
541       try {
542         const r = await fetch(url, {
543           signal: ctrl.signal,
544           headers: { 'User-Agent': 'fhcode-web/0.5.1' },
545         });
546         clearTimeout(timer);
547         return r;
548       } catch (e) {
549         clearTimeout(timer);
550         lastErr = e;
551         if (attempt < retries) await new Promise((res) => setTimeout(res, 500));
552       }
553     }
554     throw lastErr;
555   }
556
557   interface MarketSkill {
```

```
    id: string;
    name: string;
    description: string;
    source: string;
    author?: string;
    category: string;
    tags: string[];
    downloads: number;
```

```
566     installHint: string;
567     homepage: string;
568     rawUrl: string;
569   }
570
571   async function fetchClawHubSkills(keyword: string, limit: number): Promise<MarketSkill[]> {
572     try {
573       const url = `${CLAWHUB_API}?limit=${limit}${keyword ? `&q=${encodeURIComponent(keyword)}` : ''}
    `;
574       const r = await fetchWithRetry(url);
575       if (!r.ok) return [];
576       const data = (await r.json()) as { items?: Array<Record<string, any>> };
577       return (data.items || []).map((it) => ({
578         id: `clawhub:${String(it.slug || '')}`,
579         name: String(it.displayName || it.slug || ''),
580         description: String(it.summary || it.description || ''),
581         source: 'clawhub',
582         author: it.slug ? String(it.slug).split('/')[0] : undefined,
583         category: 'market',
584         tags: Array.isArray(it.topics) ? it.topics : [],
585         downloads: Number(it.stats?.downloads ?? 0),
```

```
        installHint: `npx skills add ${String(it.slug || '')}`,
        homepage: `https://clawhub.ai/${String(it.slug || '')}`,
        rawUrl: `https://clawhub.ai/${String(it.slug || '')}`,
      }));
    } catch (e) {
      console.error('[clawhub] fetch failed:', e);
      return [];
    }
  }

  async function fetchAgentFoundrySkills(keyword: string, limit: number): Promise<MarketSkill[]> {
    try {
      const r = await fetchWithRetry(AGENT_FOUNDRY_CATALOG);
      if (!r.ok) return [];
      const data = (await r.json()) as { skills?: Array<Record<string, any>> };
      const arr = Array.isArray(data.skills) ? data.skills : [];
      let list = arr.map((it) => {
        const name = String(it.name || '');
        const publicPath = String(it.public_path || '');
        const category = String(it.category || 'market');
        const rawUrl = publicPath
          ? `https://raw.githubusercontent.com/hebertzhu/agent-foundry/main/${publicPath}`
```

```
608           : `https://github.com/hebertzhu/agent-foundry`;
609         return {
610           id: `agent-foundry:${name}`,
611           name,
612           description: `Agent-Foundry 技能包 · 分类：${category}${publicPath ? ` · 源：${publicPath}` : ''}`,
613           source: 'agent-foundry',
614           author: 'hebertzhu',
615           category,
```

```
616           tags: Array.isArray(it.packs) ? it.packs : [category],
617           downloads: 0,
618           installHint: `npm i -g @agent-foundry/cli && agent-foundry install ${name}`,
619           homepage: `https://github.com/hebertzhu/agent-foundry`,
620           rawUrl,
621         };
622       });
623       if (keyword) {
624         const kw = keyword.toLowerCase();
625         list = list.filter(
626           (s) =>
627             s.name.toLowerCase().includes(kw) ||
628             s.description.toLowerCase().includes(kw) ||
629             (s.tags || []).some((t) => t.toLowerCase().includes(kw)),
630         );
631       }
632       return list.slice(0, limit);
633     } catch (e) {
634       console.error('[agent-foundry] fetch failed:', e);
635       return [];
636     }
```

```
}

app.get('/api/skills/market', async (req: Request, res: Response) => {
  const q = typeof req.query.q === 'string' ? req.query.q.trim() : '';
  const source = typeof req.query.source === 'string' ? req.query.source : 'all';
  const limit = Math.min(Number(req.query.limit) || 50, 200);
  let results: MarketSkill[] = [];
  if (source === 'clawhub' || source === 'all')
    results = results.concat(await fetchClawHubSkills(q, limit));
  if (source === 'agent-foundry' || source === 'all')
    results = results.concat(await fetchAgentFoundrySkills(q, limit));
  results.sort((a, b) => (b.downloads || 0) - (a.downloads || 0));
  res.json({ ok: true, total: results.length, skills: results.slice(0, limit) });
});

// 已安装技能持久化
const installedSkillsFile = join(homeDir, 'skills', 'installed.json');
app.post('/api/skills/install', async (req: Request, res: Response) => {
  const body = req.body as Record<string, any>;
  if (!body || !body.id || !body.name || !body.source) {
    res.status(400).json({ ok: false, error: '缺少 id / name / source 字段' });
```

```
658       return;
659     }
660     try {
661       const dir = join(homeDir, 'skills', body.id.replace(/[^a-zA-Z0-9_-]/g, '_'));
662       mkdirSync(dir, { recursive: true });
663       if (!saveJsonFile(join(dir, 'meta.json'), { ...body, installedAt: new Date().toISOString() })) {
664         res.status(500).json({ ok: false, error: '技能元数据写入失败' });
665         return;
```

```
    }
    const installed = loadJsonFile<Array<Record<string, any>>>(installedSkillsFile, []);
    if (!installed.find((s) => s.id === body.id)) {
      installed.push(body);
      if (!saveJsonFile(installedSkillsFile, installed)) {
        res.status(500).json({ ok: false, error: '已安装列表写入失败' });
        return;
      }
    }
    res.json({ ok: true, message: `技能「${body.name}」已记录为已安装` });
  } catch (e) {
    res.status(500).json({ ok: false, error: '安装失败: ' + (e as Error).message });
  }
});
app.get('/api/skills/installed', (_req: Request, res: Response) => {
  res.json({ ok: true, skills: loadJsonFile(installedSkillsFile, []) });
});

/* ========== 自动化：快捷指令集（一键发起任务） ========== */
interface AutomationRule {
  id: string;
```

```
687     name: string;
688     goal: string;
689     modelId?: string;
690     workspaceDir?: string;
691     createdAt: string;
692     runCount: number;
693     lastRunAt?: string;
694   }
695   const automationsFile = join(homeDir, 'automations.json');
696   app.get('/api/automations', (_req: Request, res: Response) => {
697     res.json({ ok: true, automations: loadJsonFile<AutomationRule[]>(automationsFile, []) });
698   });
699   app.post('/api/automations', async (req: Request, res: Response) => {
700     const body = (req.body ?? {}) as Record<string, any>;
701     const name = typeof body.name === 'string' ? body.name.trim() : '';
702     const goal = typeof body.goal === 'string' ? body.goal.trim() : '';
703     if (!name || !goal) {
704       res.status(400).json({ ok: false, error: '缺少 name 或 goal 字段' });
705       return;
706     }
707     const list = loadJsonFile<AutomationRule[]>(automationsFile, []);
```

```
708     const rule: AutomationRule = {
709       id: `auto_${Date.now()}_${randomUUID().slice(0, 4)}`,
710       name,
711       goal,
712       modelId:
713         typeof body.modelId === 'string' && body.modelId.trim() ? body.modelId.trim() : undefined,
714       workspaceDir:
715         typeof body.workspaceDir === 'string' && body.workspaceDir.trim()
```

```
716           ? body.workspaceDir.trim()
717           : undefined,
718       createdAt: new Date().toISOString(),
719       runCount: 0,
720     };
721     list.push(rule);
722     if (!saveJsonFile(automationsFile, list)) {
723       res.status(500).json({ ok: false, error: '指令保存失败（磁盘不可写）' });
724       return;
725     }
726     res.status(201).json({ ok: true, automation: rule });
727   });
728   // 注：当前 @types/express 版本缺失 delete 方法类型声明，运行时 Express 完全支持，故局部断言
729   (app.delete as (p: string, h: (req: Request, res: Response) => void) => void)(
730     '/api/automations/:id',
731     async (req: Request, res: Response) => {
732       const list = loadJsonFile<AutomationRule[]>(automationsFile, []);
733       const next = list.filter((a) => a.id !== req.params.id);
734       if (next.length === list.length) {
735         res.status(404).json({ ok: false, error: '指令不存在' });
736         return;
```

```
737       }
738       if (!saveJsonFile(automationsFile, next)) {
739         res.status(500).json({ ok: false, error: '指令删除失败（磁盘不可写）' });
740         return;
741       }
742       res.json({ ok: true });
743     },
744   );
745   app.post('/api/automations/:id/run', async (req: Request, res: Response) => {
746     const list = loadJsonFile<AutomationRule[]>(automationsFile, []);
747     const rule = list.find((a) => a.id === req.params.id);
748     if (!rule) {
749       res.status(404).json({ ok: false, error: '指令不存在' });
750       return;
751     }
752     const record = queue.submit(rule.goal, {
753       modelId: rule.modelId,
754       workspaceDir: rule.workspaceDir,
755     });
756     rule.runCount += 1;
757     rule.lastRunAt = new Date().toISOString();
```

```
758     // 任务已入队，runCount 持久化失败不阻塞响应（带 persistWarning 提示）
759     const persistWarning = !saveJsonFile(automationsFile, list);
760     res.status(201).json({
761       ok: true,
762       task: publicTask(record, true),
763       runCount: rule.runCount,
764       persistWarning: persistWarning || undefined,
765     });
```

```
});

/* ========== 模板库 ========== */
interface UserTemplate {
  id: string;
  title: string;
  category: string;
  goal: string;
  icon: string;
  builtin: boolean;
}
const BUILTIN_TEMPLATES: Omit<UserTemplate, 'builtin'>[] = [
  {
    id: 'tpl-code-gen',
    title: '编写新功能',
    category: '代码',
    icon: '🧩',
    goal: '请用 production-ready 的代码实现以下功能：\n\n（请描述功能点、输入输出、边界条件）',
  },
  {
    id: 'tpl-debug',
```

```
    title: '调试 Bug',
    category: '调试',
    icon: '🐛',
    goal: '以下代码出现异常，请定位根因并修复，附回归验证：\n\n（粘贴错误日志或现象）',
  },
  {
    id: 'tpl-refactor',
    title: '重构优化',
    category: '代码',
    icon: '♻️',
    goal: '请在不改变外部行为的前提下重构以下模块，提升可读性与性能：\n\n（粘贴代码或文件路径）',
  },
  {
    id: 'tpl-test',
    title: '生成测试',
    category: '测试',
    icon: '🧪',
    goal: '请为以下代码生成单元测试与边界用例（覆盖率优先）：\n\n（粘贴代码或文件路径）',
  },
  {
    id: 'tpl-doc',
```

```
808       title: '撰写文档',
809       category: '文档',
810       icon: '📝',
811       goal: '请撰写一份技术/产品文档，包含背景、方案、步骤与注意事项：\n\n（说明文档主题）',
812     },
813     {
814       id: 'tpl-review',
815       title: '代码审查',
```

```
816       category: '质量',
817       icon: '🔍',
818       goal: '请对以下代码做静态/设计审查，列出问题与改进建议：\n\n（粘贴代码或文件路径）',
819     },
820     {
821       id: 'tpl-explain',
822       title: '解释代码',
823       category: '文档',
824       icon: '💡',
825       goal: '请用通俗语言解释以下代码的逻辑与关键点：\n\n（粘贴代码）',
826     },
827     {
828       id: 'tpl-perf',
829       title: '性能优化',
830       category: '质量',
831       icon: '⚡',
832       goal: '请分析并优化以下代码的性能瓶颈，给出前后对比：\n\n（粘贴代码或描述场景）',
833     },
834   ];
835   const templatesFile = join(homeDir, 'templates.json');
836   app.get('/api/templates', (_req: Request, res: Response) => {
```

```
837     const user = loadJsonFile<UserTemplate[]>(templatesFile, []);
838     res.json({ ok: true, builtin: BUILTIN_TEMPLATES, user });
839   });
840   app.post('/api/templates', async (req: Request, res: Response) => {
841     const body = (req.body ?? {}) as Record<string, any>;
842     const title = typeof body.title === 'string' ? body.title.trim() : '';
843     const goal = typeof body.goal === 'string' ? body.goal.trim() : '';
844     if (!title || !goal) {
845       res.status(400).json({ ok: false, error: '缺少 title 或 goal 字段' });
846       return;
847     }
848     const list = loadJsonFile<UserTemplate[]>(templatesFile, []);
849     const tpl: UserTemplate = {
850       id: `tpl_${Date.now()}_${randomUUID().slice(0, 4)}`,
851       title,
852       category:
853         typeof body.category === 'string' && body.category.trim() ? body.category.trim() : '自定义',
854       goal,
855       icon: typeof body.icon === 'string' && body.icon.trim() ? body.icon.trim() : '📄',
856       builtin: false,
857     };
```

```
    list.push(tpl);
    if (!saveJsonFile(templatesFile, list)) {
      res.status(500).json({ ok: false, error: '模板保存失败（磁盘不可写）' });
      return;
    }
    res.status(201).json({ ok: true, template: tpl });
  });
  (app.delete as (p: string, h: (req: Request, res: Response) => void) => void)(
```

```
866     '/api/templates/:id',
867     async (req: Request, res: Response) => {
868       const list = loadJsonFile<UserTemplate[]>(templatesFile, []);
869       const next = list.filter((t) => t.id !== req.params.id);
870       if (next.length === list.length) {
871         res.status(404).json({ ok: false, error: '模板不存在或不可删除' });
872         return;
873       }
874       if (!saveJsonFile(templatesFile, next)) {
875         res.status(500).json({ ok: false, error: '模板删除失败（磁盘不可写）' });
876         return;
877       }
878       res.json({ ok: true });
879     },
880   );
881
882   /* ========== 办公助理：内置能力清单 ========== */
883   app.get('/api/office/capabilities', (_req: Request, res: Response) => {
884     res.json({
885       ok: true,
886       capabilities: [
```

```
887         {
888           id: 'doc-summary',
889           icon: '📄',
890           title: '文档摘要',
891           desc: '粘贴长文档，生成结构化摘要与要点',
892           prompt: '请对以下文档做摘要，提取 3-5 个核心要点并列出待办：\n\n',
893         },
894         {
895           id: 'doc-translate',
896           icon: '🌐',
897           title: '中英互译',
898           desc: '技术文档/代码注释翻译',
899           prompt: '请将以下内容准确翻译（保留代码与术语）：\n\n',
900         },
901         {
902           id: 'doc-rewrite',
903           icon: '✍️',
904           title: '润色改写',
905           desc: '把草稿改写为正式/简洁风格',
906           prompt: '请润色以下文本，使其更专业简洁：\n\n',
907         },
```

```
908         {
909           id: 'meeting-notes',
910           icon: '📋',
911           title: '会议纪要',
912           desc: '从聊天/录音转写生成行动项',
913           prompt: '请从以下记录中提取：决策、负责人、截止时间、待办：\n\n',
914         },
915         {
```

```
916           id: 'email-draft',
917           icon: '✉',
918           title: '邮件起草',
919           desc: '按要点生成工作邮件',
920           prompt: '请起草一封邮件，主题/背景/诉求如下：\n\n',
921         },
922         {
923           id: 'excel-formula',
924           icon: '📊',
925           title: '表格公式',
926           desc: '描述需求生成 Excel/Sheets 公式',
927           prompt: '请写出实现以下需求的表格公式并解释：\n\n',
928         },
929       ],
930     });
931   });
932
933   /* ========== 大模型配置（三重加密：apiKey AES-256-GCM 落盘加密 + RSA 加密传输） ========== */
934   const modelsFile = join(homeDir, 'models.json');
935   interface ModelConfig {
936     id: string;
```

```
937     name: string;
938     apiBase: string;
939     apiKey: string; // 存储层为 AES-256-GCM 密文（v1:...），对外永不明文返回
940     reasoning?: string;
941     default?: boolean;
942   }
943   function loadModels(): ModelConfig[] {
944     const list = loadJsonFile<ModelConfig[]>(modelsFile, []);
945     return Array.isArray(list) ? list : [];
946   }
947   /** 解析真实（解密）apiKey，供任务执行器使用 */
948   function resolveApiKey(m: ModelConfig): string {
949     if (!m.apiKey) return '';
950     return isEncrypted(m.apiKey) ? decryptText(m.apiKey, masterKey) : m.apiKey;
951   }
952   /** 对外脱敏视图：任何接口都不返回密钥（明文或密文） */
953   function publicModel(m: ModelConfig): ModelConfig {
954     return { ...m, apiKey: '' };
955   }
956   function saveModels(list: ModelConfig[]): boolean {
957     // 明文 key 加密后再落盘（已加密的跳过，避免二次加密）
```

```
958     const encList = list.map((m) =>
959       m.apiKey && !isEncrypted(m.apiKey) ? { ...m, apiKey: encryptText(m.apiKey, masterKey) } : m,
960     );
961     const ok = saveJsonFile(modelsFile, encList);
962     // 同步更新任务队列的模型配置，使用解密后的真实 key（即使落盘失败也更新内存态）
963     queue.setModelProviders(encList.map((m) => ({
964       id: m.id,
965       type: 'openai-compatible' as const,
```

```
966       baseURL: m.apiBase,
967       apiKey: resolveApiKey(m),
968       model: m.name,
969     }));
970     return ok;
971   }
972   // 初始化模型提供列表，并注册到任务队列
973   const initialModels = loadModels();
974   queue.setModelProviders(initialModels.map((m) => ({
975     id: m.id,
976     type: 'openai-compatible' as const,
977     baseURL: m.apiBase,
978     apiKey: resolveApiKey(m),
979     model: m.name,
980   })));
981   // 第二重（通信层）：向客户端下发 RSA 公钥，用于加密敏感参数（如模型 API Key）传输
982   app.get('/api/security/public-key', (_req: Request, res: Response) => {
983     res.json({ ok: true, publicKey: rsaKeys.publicKey, algorithm: 'RSA-OAEP-2048-SHA256' });
984   });
985   app.get('/api/models', (_req: Request, res: Response) => {
986     const list = loadModels().map(publicModel);
```

```
  res.json({ ok: true, models: list, defaultId: (list.find((m) => m.default) || {}).id || null });
});
app.post('/api/models', (req: Request, res: Response) => {
  const body = req.body as Record<string, any>;
  const name = typeof body?.name === 'string' ? body.name.trim() : '';
  const apiBase = typeof body?.apiBase === 'string' ? body.apiBase.trim() : '';
  let apiKey = typeof body?.apiKey === 'string' ? body.apiKey : '';
  // 支持 RSA 公钥加密传输的密钥（App 端使用，防窃听/防中间人截取明文 Key）
  if (typeof body?.apiKeyEnc === 'string' && body.apiKeyEnc) {
    const dec = rsaDecrypt(body.apiKeyEnc, rsaKeys.privateKey);
    if (dec) apiKey = dec;
  }
  const reasoning = typeof body?.reasoning === 'string' ? body.reasoning : '';
  if (!name) {
    res.status(400).json({ ok: false, error: '请填写模型名称' });
    return;
  }
  const list = loadModels();
  const id = typeof body?.id === 'string' && body.id.trim() ? body.id.trim() : randomUUID();
  const idx = list.findIndex((m) => m.id === id);
  let saved: ModelConfig;
```

```
1008    if (idx >= 0) {
1009      // 编辑场景：未提供新 key 则保留原密钥（前端不再回填明文）
1010      const cfg: ModelConfig = {
1011        id, name, apiBase,
1012        apiKey: apiKey || list[idx].apiKey,
1013        reasoning,
1014      };
1015      list[idx] = { ...list[idx], ...cfg };
```